C

A simple reference of a complex language

(French version v1.0)

Medard Rieder (medard.rieder@hevs.ch)
© 2019 HES-SO Valais - Systèmes Industriels

TABLE DES MATIERES

1 Introduction 1
1.1 Le langage C 1

2 Structure d'un programme C 2
2.1 Forme générale d'un programme C 2
Directives de pré compilation 2
Déclarations globales 2
Fonctions 3
2.2 Symboles d'un programme C 5
Commentaires 5
Identificateurs 5
Mots-clés 6
Littéraux 6
Exemple 8

3 Types de données 9
3.1 Types fondamentaux 9
Types entiers 10
Types réels, à virgule flottante 10
Type booléen 10
Types caractères 10
Type `void` 11
Type énumération 11
3.2 Types dérivés 12
3.3 Types synonymes 12

4 Instructions simples 13
4.1 Déclaration de constantes 13
4.2 Déclaration de variables 14
4.3 Affectation simple 14
4.4 Instructions d'entrées/sorties 15
Entrées/sorties C 15

5 Expressions et opérateurs 18
5.1 Opérateurs 18
Opérateurs d'affectation 18
Opérateurs arithmétiques 19
Opérateurs relationnels 20
Opérateurs logiques 20
Incrémentation et décrémentation 20
Opérateurs de décalage 21
Opérateurs logiques bit-à-bit 22
Ordre de précédence des opérateurs 22
5.2 Conversions de types 23
5.3 Fonctions mathématiques 25

6 Instructions de contrôle 26
6.1 Séquence ou bloc 26
6.2 Sélections 26
Instruction `if` 26
Instruction `switch` 28
6.3 Itérations 30
Instruction `while` 30
Instruction `do...while` 31
Instruction `for` 32
6.4 Sauts 34
Instruction `break` 34
Instruction `continue` 34
Instruction `return` 34
Instruction `goto` 34

7 Fonctions 35
7.1 Décomposition algorithmique 35
7.2 Déclaration d'une fonction 36
7.3 Définition d'une fonction 37
7.4 Appel d'une fonction 38
7.5 Variables locales et variables globales 39
7.6 Passage de paramètres 42
Passage par valeur 42
Passage par pointeur (référence) 42
7.7 Fonctions `inline` 43
7.8 Type du résultat d'une fonction 43

8 Pointeurs 44
8.1 Pointeurs 44
8.2 Déclaration de pointeurs 44
Utilisation de pointeurs 44
Pointeurs et constantes 46

9 Tableaux 47
9.1 Notion de tableau 47
9.2 Tableaux unidimensionnels 47
Déclaration 47
Initialisation 48
Accès aux éléments du tableau 48
9.3 Tableaux multidimensionnels 49
Déclaration 49
Initialisation 49
Accès aux éléments du tableau 49
9.4 Tableaux et pointeurs 50
9.5 Paramètres tableaux 51
9.6 Opérateur `sizeof` 53

10 Chaînes de caractères 54
10.1 Nature d'une chaîne de caractères 54
Littéraux chaînes de caractères 54
Variables chaînes de caractères 54
Pointeurs sur chaînes de caractères 56
10.2 Opérations sur les chaînes 56
Lecture d'une chaîne 56
10.3 Affichage d'une chaîne 57
Copie de chaînes 57
Comparaison de chaînes 57
Longueur d'une chaîne 58
Fonctions de la libraire `<string.h>` 58

11 Structures et unions 59
11.1 Structures 59
Déclaration de structures 59
11.2 Accès aux membres d'une structure 60
Utilisation des structures 60
11.3 Champs de bits 62
Déclaration de champs de bits 62
Utilisation de champs de bits 63
11.4 Unions 63
Déclaration d'une union 63
Utilisation d'unions 64

12 Allocation dynamique 65
12.1 Allocation dynamique de mémoire 65
12.2 Allocation dynamique en C 65
12.3 Exemples 65

13 Classes de stockage 69
13.1 Définitions 69
Portée 69
Durée de vie 70
Emplacement mémoire 70
13.2 Classes de stockage 71
Spécificateur `extern` 71
Le spécificateur `auto` 71
Le spécificateur `static` 72
Spécificateur `register` 72
13.3 Exemples 73

 © HES-SO VS / rim 2023, v1-0

14 Directives de prétraitement 79
14.1 Forme d'une directive 79
14.2 `#define` (définition) 80
Définition de constantes *80*
Définition de macros *81*
14.3 `#undef` (suppression de définition) 82
14.4 `#include` (inclusion de fichier) 82
14.5 Compilation conditionnelle 83
Clause `if` *83*
Clause `elif` *83*
Clause `else` *84*
14.6 `#line` (contrôle des lignes) 85
14.7 `#error` (contrôle du message d'erreur) 85
14.8 `#pragma` 86

1 Introduction

Cette courte introduction situe le langage C par rapport aux autres langages de programmation.

1.1 Le langage C

Le langage de programmation C a été développé à partir de 1970 par Dennis Ritchie aux Laboratoires Bell AT&T. Les spécifications du langage ont été publiées en 1978 par Ritchie et Kernighan.

Le langage C est un successeur des langages BCPL et B. Sa création avait pour objectif initial de remplacer le codage en langage assembleur pour la plupart des tâches de programmation système. Bien qu'appartenant à la génération des langages de programmation de haut niveau, plus ou moins structurés, C est cependant un langage permissif, puisqu'il n'y a pas ou peu de contrôle de type.

Son succès est issu en majeure partie du fait qu'il est le langage d'implantation du système *Unix*. Il offre également une grande portabilité, les fonctionnalités liées à la machine étant en général reléguées dans des bibliothèques.

© HES-SO VS / rim 2023, v1.0

2 Structure d'un programme C

L'objectif de ce chapitre est de présenter les différentes parties composant un programme C simple.

2.1 Forme générale d'un programme C

Voici le plus petit programme C, qui affiche le texte "Hello world!" à l'écran :

Exemple 2.1

```c
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

Figure 1 : Un simple programme en C

En règle générale, tout programme C est composé, dans l'ordre, de :

- directives de pré compilation
- déclarations globales
- suite de fonctions

Parmi la suite de fonctions doit figurer la fonction particulière appelée `main()` qui est l'équivalent du programme principal. Elle est automatiquement exécutée en premier et peut se situer n'importe où dans la suite.

Remarques

La valeur `int` renvoyée par `main()` représente la valeur renvoyée par le programme au système. En cas d'exécution réussie, cette valeur vaut 0 et il n'est pas nécessaire de la renvoyer explicitement.

Directives de pré compilation

Elles permettent entre autres d'inclure dans le programme des fichiers en-têtes, c'est-à-dire des fichiers source, modules ou librairies. Chaque directive d'inclusion commence par le symbole `#` immédiatement suivi du mot `include` suivi du nom du fichier à inclure.

- lorsque le fichier appartient à la bibliothèque C, le nom du fichier, avec l'extension, est entouré des signes < et >.
  `#include <stdio.h>`
- les autres fichiers en-têtes sont notés avec leur suffixe et délimités de guillemets.
  `#include "myfile.h"`

Déclarations globales

Chaque nom ou identificateur doit être déclaré avec son type pour pouvoir être utilisé dans un programme C.

Dans cette partie, il est possible de déclarer les éléments globaux tels que constantes, variables, ou fonctions. Un élément global est un élément dont la portée s'étend du point de la déclaration de l'élément jusqu'à la fin du fichier où la déclaration figure. La portée d'un élément est la partie du programme où cet élément peut être utilisé.

```
static const float PI = 3.14159f;
static const int MAX = 100;
```

Fonctions

Une fonction est un sous-programme. Sa définition doit contenir :

- une en-tête de fonction, composé du nom de la fonction suivi d'une liste de paramètres optionnels avec leurs types, délimitée par des parenthèses; le nom de la fonction est précédé du type du résultat retourné s'il y en a un ou, sinon, du mot `void`;
- le corps de la fonction, délimité par les accolades `{` et `}`, contenant les déclaration et les instructions; on distingue principalement :
- les déclarations des variables locales;
- les instructions simples;
- les structures de contrôle ou instructions de contrôle.

Le point-virgule est utilisé en général comme délimiteur et terminaison d'une instruction. `int main(void)` est une fonction.

Instructions simples

Attention, les instructions simples C regroupent non seulement les instructions qualifiées habituellement de simples comme l'affectation mais également des déclarations et expressions.

On distingue donc :

- les déclarations
- les instructions-expressions
- les instructions de branchement

Dans la catégorie d'instructions-expressions figurent par exemple l'affectation et l'appel de fonction.

L'affectation permet de donner ou d'affecter une valeur à une ou plusieurs variables. Le symbole courant utilisé pour l'affectation est le signe =, appelé opérateur d'affectation.

L'appel de fonction peut être un appel à une fonction de bibliothèque, par exemple pour une instruction d'entrée/sortie, ou un appel à une autre fonction du programme. L'en-tête de la fonction doit toujours être connu avant son utilisation. Dans le contexte d'un appel à une autre fonction du programme, on fera la distinction entre déclaration et définition : avant l'appel à la fonction on déclare son en-tête, qui sera désigné par le terme de prototype ; puis la définition de la fonction, en-tête y compris, pourra être placée n'importe où dans le programme.

 © HES-SO VS / rim 2023, v1.0

Exemple 2.2

```c
#include <stdio.h>

static const float PI = 3.14159f;
static const int MAX = 100;

void f();

int main(void)
{
    int i,j;
    i = 2;
    i = j = 5;
    f();
    return 0;
}

void f()
{
    int a = 1;
    printf("%i\n",a);
}
```

Figure 2 : Structure d'un simple programme en C

Remarque
En C, il n'y a pas d'imbrication de fonctions. Mais des fonctions peuvent être déclarés à l'intérieur d'une autre fonction et deviennent avec ça "privées", donc seulement visibles dans ces fonctions qui contiennent leures déclarations.

Structures de contrôle

Les structures de contrôle sont les instructions permettant de contrôler le déroulement du programme. On distingue :

- l'instruction composée ou bloc (séquence)
- les instructions de sélection
- les instructions d'itération

Elles seront expliquées en détail dans un chapitre ultérieur.

 © HES-SO VS / rim 2023, v1.0

2.2 Symboles d'un programme C

Il existe cinq classes de symboles : identificateurs, mots-clés, littéraux ou constantes, opérateurs et ponctuateurs ou séparateurs. Les commentaires, espaces, tabulations, etc. sont regroupés sous le nom de séparateurs.

Commentaires

L'importance des commentaires dans un programme n'est plus à démontrer. Ceux-ci peuvent être introduits de deux façons :

- soit en entourant le commentaire des symboles `/*` et `*/`
  sur une ou plusieurs lignes, sans imbrication
- soit en débutant le commentaire par `//`
  dans ce cas le commentaire est terminé à la fin de la ligne

Identificateurs

Un identificateur est le nom que le programmeur donne à une constante, variable, type, fonction, etc.

Un identificateur est formé d'un nombre quelconque de lettres majuscules ou minuscules non accentuées, de chiffres, ainsi que du caractère souligné considéré comme une lettre. Le premier caractère doit être une lettre. Les majuscules et minuscules sont considérées comme des caractères différents.

Il n'y a pas de style ou convention unique pour le choix et l'écriture d'identificateurs, mais il est recommandé d'en adopter un. Voici quelques recommandations :

- les noms doivent être suffisamment significatifs et refléter la signification de l'entité plutôt que son implémentation;
- si un identificateur est un nom composé, le deuxième nom commencera par une majuscule ou bien sera séparé du premier par le caractère souligné;
  `someSymbol, another_symbol`
- les identificateurs de constantes sont écrits en majuscule;
- les identificateurs des types définis par l'utilisateur commencent par une majuscule;
- les autres identificateurs sont écrits en minuscule;
- il est **déconseillé** de commencer un identificateur par `_` ou d'utiliser le double souligné `__` ceci afin d'éviter des interférences avec les noms utilisés dans les librairies C.

Mots-clés

Les identificateurs suivants constituent les mots-clés du langage, réservés à des fins spéciales.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while
_Bool (C99)	_Imaginary (C99)	restrict (C99)	_Complex (C99)
inline (C99)			

Tableau 1 : Mots clés C

Littéraux

Les littéraux sont des constantes associées aux différents types de base du langage.

On distingue :

- les constantes entières : toute valeur ne commençant pas par le chiffre `0` est considérée comme du décimal. Une valeur précédée du chiffre `0` est interprétée comme étant de l'octal ; une valeur précédée de `0x` ou `0X` est interprétée comme étant de l'hexadécimal

  ```
  129   0423   0x54a
  ```

- les constantes réelles, en virgule flottante

  ```
  123.48   .123.48f   .12.327e+1 .28   .236  .4e2
  ```

- les constantes caractères
  Une constante caractère est formée d'un caractère unique écrit entre apostrophes :

  ```
  'a'
  ```

  Certains caractères non affichables comme le retour à la ligne, nouvelle page ou sonnerie, ainsi que les guillemets, l'apostrophe, le point d'interrogation et la barre oblique inversée peuvent être représentés par des séquences d'échappement. Celles-ci commencent toujours par la barre oblique inversée suivie d'un ou plusieurs caractères ayant un rôle particulier : par exemple `\n` est la séquence d'échappement ou caractère de contrôle correspondant au retour de ligne.

 © HES-SO VS / rim 2023, v1.0

Bien qu'une séquence d'échappement soit matérialisée par deux ou plusieurs caractères, elle est considérée comme une constante de type caractère, puisqu'elle représente un code unique, et sera par conséquent mise entre apostrophes :

```
'\n'
```

- les constantes chaînes de charactères
  Une constante chaîne de caractères est formée d'un nombre quelconque de caractères encadrés par des guillemets.
  Une chaîne de caractères peut contenir des séquences d'échappement, celles-ci seront alors notées sans apostrophes à l'intérieur de la chaîne délimitée par les guillemets. Par exemple :

```
"this is a string litteral"
"\nLine 1\nLine 2"
```

Séquence d'échap.	Signification
\b	retour arrière
\f	saut de page
\n	saut de ligne
\r	retour chariot
\t	tabulation horizontale
\v	tabulation verticale
\\	barre oblique inversée
\'	guillemet simple ou apostrophe
\"	guillemet double

Tableau 2 : Principales constantes caractères

 © HES-SO VS / rim 2023, v1.0

Exemple

L'exemple ci-après montre deux versions d'un petit programme C qui lit la valeur du rayon d'un cercle, calcule puis affiche la surface de ce cercle. La première est composée d'une seule fonction, `main`, tandis que l'autre version utilise une deuxième fonction, `areaCircle`, afin de montrer la déclaration du prototype et la définition de la fonction.

Exemple 2.3

```c
/* Area of a circle */


#include <stdio.h>                  // access to library
static const float PI = 3.14159f;   // global constant

int main()                          // header of main function
{                                   // beginning of the body
    float radius, area;             // declar. of variables

    printf("radius = ");            // display to screen
    scanf("%f",&radius);            // reading of radius
    area = PI * radius * radius;    // calculate the area

    printf("Area is %f\n",area);    // display the result
}
```

Figure 3 : Surface d'un cercle, version 1

Exemple 2.4

```c
/* Area of a circle */

#include <stdio.h>                  // access to library
static const float PI = 3.14159f;   // global constant

int main()                          // header of main function
{                                   // body of main function
    float radius;                   // declare radius

    float areaCircle(float r);      // function prototype, note:
                                    // areaCircle is only visible
                                    // in the main() function
    printf("radius = ");            // display to screen
    scanf("%f",&radius);            // reading of radius

                                    // the call of the function
                                    // areaCircle is here
    printf("Area = %f\n",areaCircle(radius));
}

// Definition of the function areaCircle
float areaCircle(float r)           // header of the function
{                                   // body of the function
    float a;                        // declaration of a
    a = PI * r * r;
    return a;                       // return the result
}
```

Figure 4 : Surface d'un cercle, version 2

3 Types de données

Puisque C est un langage typé, chaque nom ou identificateur est associé à un type de données. Ce type détermine la nature et les opérations permises pour l'entité représentée par ce nom.

C possède une grande variété de types de données, répartis en deux catégories :

- les types fondamentaux
- les types dérivés

Ce chapitre présente les types fondamentaux en détail et énumère simplement les types dérivés qui feront chacun l'objet d'un chapitre séparé.

3.1 Types fondamentaux

Les types fondamentaux regroupent :

- les types entiers
- les types réels
- le type booléen
- les types caractères
- le type `void`
- le type énumération

Parmi les types fondamentaux, le C possède des types simples prédéfinis, qui correspondent aux unités de stockage les plus communes d'un ordinateur.

Les principaux sont :

- `int` (entier)
- `double` (réel)
- `bool, _Bool` (booléen) (`bool` demande l'inclusion de stdbool.h)
- `char` (caractère)

Ces types possèdent des variantes et peuvent être utilisés au moyen du spécificateur de type correspondant.

- Les types entier, booléen et caractère sont appelés type intégral.
- Les types intégral et réel constituent les types arithmétiques.

© HES-SO VS / rim 2023, v1.0

Types entiers

Ils sont utilisés pour représenter des nombres entiers de tailles différentes.

La norme C ne définit pas l'étendue des nombres et le nombre de bits utilisés pour le codage dépend de la plateforme.

Spécificateur de type	Signification
`int`	entier ordinaire
`short int`	entier court
`unsigned int`	entier non-signé
`long int`	entier long
`unsigned long`	entier long non-signé

Tableau 3 : Types entiers

Types réels, à virgule flottante

L'ensemble des valeurs d'un type réel est un sous-ensemble des nombres réels au sens mathématique. Le C possède trois types réels prédéfinis.

Spécificateur de type	Précision
`float`	simple précision
`double`	double précision
`long double`	précision étendue

Tableau 4 : Types réels

La signification de la précision des nombres à virgule flottante dépend de l'implémentation.

Type booléen

Le type booléen `bool` offre deux valeurs logiques possibles :

- `true` pour vrai
- `false` pour faux

Par définition, `true` correspond à la valeur entière `1`, et `false` à la valeur entière `0`. Pour pouvoir utiliser ce type, il faut inclure la libraire `<stdbool.h>`. Il existe aussi le type `_Bool`. Mais il ne connaît pas des littéraux `false` ou `true`. `0` est false, toute autre valeur est `true`.

Types caractères

Le type caractère `char` représente le jeu de caractères défini par l'implémentation, en général la suite des 256 caractères ASCII, où chaque caractère est représenté par un octet de 8 bits.

Lorsque l'on affecte un caractère à une variable de type caractère, on écrira le caractère entre apostrophes :

```
letter = 'A';
```

Spécificateur de type	Etendue
`char`	défini par l'implémentation
`signed char`	-128...127
`unsigned char`	0...255

Tableau 5 : Types caractères

Type `void`

Le type `void` spécifie un ensemble de valeurs vide. Il est utilisé comme type de retour pour les fonctions ne retournant pas de résultat. Il n'est pas possible de déclarer un objet de type `void`. Cependant, `void` peut servir de type de base pour des pointeurs sur des objets de type inconnu.

Type énumération

Le type énumération appartient à la catégorie des types entiers.

Il permet à l'utilisateur de définir un nouveau type de données dont les valeurs possibles sont constituées d'une liste de constantes désignées par des identificateurs. La déclaration d'un type énumération commence par le mot-clé `enum` suivi du nom du type et de l'énumération elle-même entre accolades. L'énumération doit spécifier un domaine de valeurs discrètes par nomination de chaque valeur à l'aide d'un identificateur appelé énumérateur.

Syntaxe

```
enum identifierType   (enumerator, enumerator,..., enumerator};
```

Voici la déclaration d'un type énumération appelé `color` :

```
enum Color {PURPLE, BLUE, GREEN, YELLOW, RED};
```

Chaque énumérateur est représenté par une valeur constante. La valeur de chaque énumérateur est par défaut son numéro d'ordre dans la liste. Ainsi, dans l'exemple, l'énumérateur `PURPLE` a la valeur entière `0`, `BLUE` la valeur `1`, `GREEN` la valeur `2` et ainsi de suite.

Il est possible d'affecter une valeur spécifique à un énumérateur lors de la déclaration d'énumération :

```
enum Color { PURPLE, BLUE =4, GREEN, YELLOW, RED };
```

L'énumérateur `purple` a la valeur `0`, `blue` la valeur `4`, `green` la valeur `5` etc.

Les noms des énumérateurs doivent être distincts de ceux des variables et constantes ordinaires et des autres énumérateurs dans la même portée. Cependant, les valeurs des énumérateurs n'ont pas besoin d'être distinctes.

3.2 Types dérivés

A partir des types fondamentaux, il est possible de construire une grande variété de types dérivés en se servant des opérateurs de déclaration `*`, `&`, `[ ]` et `( )`, et de mots-clés.

La majorité de ces types sont les outils de construction des types utilisateurs. Ils seront expliqués en détail ultérieurement.

Dans cette catégorie, on distingue :

- les fonctions
- les pointeurs
- les tableaux
- les structures et unions

3.3 Types synonymes

Un type synonyme est un alias d'un type existant. Le mot-clé `typedef` permet de renommer un type existant, fondamental ou dérivé, à l'aide d'un nom choisi comme synonyme.

Syntaxe

```
typedef  identifierType  newIdentifier;
```

Par exemple :

```
typedef int integer;
```

Un type synonyme n'introduit pas un nouveau type :

```
typedef int integer;
integer distance;
int meter;
```

Dans l'exemple ci-dessus, les deux variables `distance` et `meter` sont du même type, `int`.

4 Instructions simples

Ce chapitre est consacré à l'étude de quelques instructions simples, en particulier :

- des déclarations, dont
  - la déclaration de constantes
  - la déclaration de variables
- des instructions-expressions, dont
  - l'affectation simple
  - les instructions d'entrées/sorties

4.1 Déclaration de constantes

Il est très fréquent d'utiliser dans un programme une même valeur, comme π en mathématiques. Une telle donnée, dont la valeur ne change pas au cours du programme, s'appelle une constante. Comme beaucoup de langages, C permet de donner un nom ou identificateur à une constante, en la déclarant dans le programme à l'aide du mot-clé `const`. La référence à cette valeur se fera ensuite par l'intermédiaire de son nom.

Syntaxe

```
const specifierType identifier = value;
```

Par convention, l'identificateur de la constante s'écrit en majuscules.

Il est possible de définir des constantes symboliques pour tous les types, en particulier les types fondamentaux :

- constante de type entier
  ```
  const int NUMBER = 1;
  ```
- constante de type réel
  ```
  const float PI = 3.14156f;
  ```
- constante de type caractère
  ```
  const char LETTER = 'a';
  ```

  Certaines de ces constantes peuvent être des caractères de contrôle
- constante booléenne
  ```
  const bool TORF = true;
  ```

 © HES-SO VS / rim 2023, v1.0

4.2 Déclaration de variables

Une variable est une donnée dont le contenu peut varier au cours de l'exécution du programme. En C, chaque variable doit être déclarée en précisant d'abord son type puis son nom.

Syntaxe

```
storageClassSpecifier specifierType identifier;
```

Remarque

Le spécificateur de classe de stockage est un élément optionnel que nous n'utiliserons pas pour l'instant. Les différentes classes de stockage feront l'objet d'un chapitre ultérieur.

Voici quelques exemples de déclarations :

```
double unitPrice, amount;
char letter;
long int quantity;
```

Les variables peuvent être initialisées lors de leur déclaration :

```
double unitPrice = 13.5;
char letter = 'a';
```

La déclaration d'une variable doit être faite avant toute utilisation de celle-ci. Puisqu'une déclaration est considérée comme une instruction, elle peut intervenir n'importe où dans le programme. Chaque déclaration introduit un nom dans une portée, cela signifie que ce nom ne peut être utilisé que dans une partie spécifique du programme. Ainsi pour un nom déclaré dans une fonction, sa portée s'étend depuis le point de déclaration jusqu'à la fin du bloc. En C, les déclarations sont placées au début du bloc dans lequel elles existent.

4.3 Affectation simple

L'instruction d'affectation permet de donner ou d'affecter le résultat d'une expression à une ou plusieurs variables.

Le symbole utilisé pour l'affectation simple est le signe =. Il est considéré comme un opérateur. Il existe d'autres opérateurs d'affectation qui seront décrits dans le chapitre suivant.

```
int a, b;
b = 10;
a = b;
```

4.4 Instructions d'entrées/sorties

Par instructions d'entrées/sorties, en abrégé E/S, on désigne les instructions permettant à un programme de communiquer avec son environnement, par exemple lecture de données, affichage de données ou opérations sur fichiers.

Entrées/sorties C

Un programme C peut aussi utiliser des instructions d'E/S. Celles-ci sont obtenues par des appels à des fonctions de bibliothèques, en particulier la bibliothèque `<stdio.h>`.

Lecture de données

La lecture de données entrées au clavier se fait par la fonction standard `scanf`.

Syntaxe

```
scanf(formatString,arg1,arg2,arg3,....argn);
```

Cet appel contient une liste de paramètres placés entre parenthèses. Ces paramètres sont constitués de la chaîne de format pour la ou les valeurs que l'on désire lire, et de la liste d'arguments constitués chacun du caractère `&` et de l'identificateur de variable. Le caractère `&` représente l'adresse mémoire de la variable.

```
scanf("%f",&price);
```

Après avoir introduit au clavier par exemple la valeur `1.75` et avoir appuyé sur la touche `<Enter>`, la variable `price` contiendra la valeur `1.75`. Dans ce cas, `price` est une variable de type réel (`%f`).

```
scanf("%d %f %f",&quantity,&price,&discount);
```

Ici, on introduit au clavier trois valeurs (une entière, deux réelles) sur la même ligne, séparées par un espace, puis on appuie sur `<Enter>`.

Les chaînes de formats avec spécificateurs de types sont expliquées dans le tableau 7.

Affichage de données

Pour afficher des données à l'écran, le C offre la fonction standard `printf`.

Syntaxe

```
printf (formatString,arg1,arg2,arg3,....argn);
```

La liste de paramètres contient une chaîne de format suivi de la ou des données à afficher. Une donnée peut être une variable ou une expression.

```
printf("%f %d",price,quantity);
```

Le résultat de l'exécution de cette instruction sera l'affichage de la valeur de la variable `prix` et de la valeur de la variable `quantity`, séparées par un espace, sur la même ligne. Le curseur reste positionné sur la même ligne, juste après la valeur de `quantity`.

```
printf("Invoice amount = %f",amount);
```

Affichage de la chaîne de caractères `Montant de la facture =` suivie de la valeur de la variable `amount`.

```
printf("%f\n",price * quantity);
```

Ici, c'est le résultat de la multiplication de `price` par `quantity` qui sera affiché. Puis le curseur se positionne au début de la ligne suivante, à cause du caractère de contrôle `\n`.

```
printf("\n");
```

Si le curseur est au début de la ligne, cette instruction permet de sauter une ligne. Sinon, elle provoque simplement le positionnement du curseur au début de la ligne suivante.

Chaînes de formatage

Une chaîne de formatage est une chaîne de caractères entre guillemets, contenant pour chaque valeur à lire ou à afficher une spécification de type précédée du signe `%`. Une chaîne de format peut également contenir des caractères de contrôle.

Charactère de contrôle	Sortie	Exemple
d *ou* i	Signed decimal integer	392
u	Unsigned decimal integer	7235
o	Unsigned octal	610
x	Unsigned hexadecimal integer	7fa
X	Unsigned hexadecimal integer (uppercase)	7FA
f	Decimal floating point, lowercase	392.65
F	Decimal floating point, uppercase	392.65
e	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
E	Scientific notation (mantissa/exponent), uppercase	3.9265E+2
g	Use the shortest representation: %e or %f	392.65
G	Use the shortest representation: %E or %F	392.65
a	Hexadecimal floating point, lowercase	-0xc.90fep-2
A	Hexadecimal floating point, uppercase	-0XC.90FEP-2
c	Character	a
s	String of characters	sample
p	Pointer address	b8000000
n	Nothing printed. The corresponding argument must be a pointer to a signed int. The number of characters written so far is stored in the pointed location.	
%	A % followed by another % character will write a single % to the stream.	%

Tableau 6 : Spécification de type dans les chaînes de format

Remarque

Lors de la lecture de données, la chaîne de format ne peut pas contenir de texte.

La chaîne de format peut contenir un texte lors de l'affichage de données. Pour améliorer la lisibilité ou pour, par exemple, aligner en colonnes des valeurs, il est possible de

préciser dans la chaîne de format, entre le signe `%` et la spécification de type, la largeur du champ d'affichage.

```
printf("Here is the quantity : %10d",quantity);
```

Ici, la largeur est de dix caractères ; la valeur sera justifiée à droite.

```
printf("%10.2f",amount);
```

Dans le cas d'affichage de valeurs de type réel, la largeur du champ d'affichage est déterminée par deux nombres. Le premier donne le nombre total de positions réservées pour l'affichage, point y compris, et le deuxième donne le nombre de décimales.

Si la largeur du champ d'affichage est précédée du signe –, la valeur est justifiée à gauche. Lorsque la largeur prévoit plus de positions que nécessaire, des espaces sont ajoutés. Si la largeur est trop petite pour la valeur, elle est ignorée.

Exemple 4.1

```c
#include <stdio.h>

int main()
{
    int integer;
    float value;
    char letter;

    integer = 435;
    value = 136.64f;
    letter = 'A';
    printf("%d\n", integer);     //435
    printf("%5d\n", integer);    //  435
    printf("%-5d\n" ,integer);   //435
    printf("%5d\n", -integer);   // -435
    printf("%2d\n", integer);    //435
    printf("%f\n", value);       //136.639999
    printf("%10.2f\n", value);   //    136.64
    printf("%2.6f\n", value);    //136.639999
    printf("%6c\n", letter);     //     A
}
```

Figure 5 : Utilisation de formats différents

5 Expressions et opérateurs

Une expression est une combinaison d'opérateurs et d'opérandes tels que variables, constantes ou fonctions. A l'intérieur d'une même expression, les opérandes doivent être de même type ou de types compatibles.

```
grossPrice + 8;
```

`grossPrice` est le premier opérande de l'expression, `+` est l'opérateur et `8` le deuxième opérande.

Les nombreux opérateurs du C ainsi que les conversions de types sont expliqués dans ce chapitre.

5.1 Opérateurs

Les principaux groupes d'opérateurs C sont :

- les opérateurs d'affectation
- les opérateurs arithmétiques
- les opérateurs relationnels
- les opérateurs logiques
- les opérateurs d'incrémentation et de décrémentation
- les opérateurs de décalage
- les opérateurs logiques bit-à-bit

Opérateurs d'affectation

L'affectation permet de stocker ou mémoriser le résultat d'une expression dans une variable. L'opérateur d'affectation est le signe = et se lit est égal à.

Il existe en C différentes affectations :

- L'affectation simple.
  L'opérateur d'affectation n'apparaît qu'une fois dans l'expression.
  `netPrice = grossPrice + 8;`
  Le résultat de l'expression `grossPrice + 8` est affecté à la variable `netPrice`.
- L'affectation multiple.
  L'opérateur d'affectation apparaît plusieurs fois dans l'expression.
  `a = b = c = 12.4;`
  Les opérateurs d'affectation sont associatifs à droite. Dans cet exemple, `c` prend la valeur `12.4`, puis `b` prend la valeur de `c`, puis `a` prend la valeur de `b`.

 © HES-SO VS / rim 2023, v1.0

- Les affectations composées.
  Dans les affectations du style `x=x op y`, où une même variable apparaît à la fois à gauche et à droite de l'opérateur =, on utilise une notation abrégée à l'aide de l'opérateur d'affectation `op=`. Ainsi, par exemple, l'expression `i=i+k` peut s'écrire `i+=k`. Cette dernière opération est plus rapide car l'emplacement mémoire réservé à `i` est modifié sans utiliser d'emplacement intermédiaire pour l'évaluation.

Opérateur	Exemple	Équivalent à
=	`x = y`	
*=	`x *=y`	`x = x * y`
/=	`x /= y`	`x = x / y`
+=	`x += y`	`x = x + y`
−=	`x −=y`	`x = x − y`
%=	`x %= y`	`x = x % y`

Tableau 7 : Opérateurs d'affectation

Tous les opérateurs d'affectation demandent que l'opérande situé à gauche de l'opérateur soit une *lvalue* modifiable c'est-à-dire une variable ou une référence à un emplacement-mémoire. A l'instar des fonctions, une expression retourne un résultat dont la valeur est celle de la *lvalue*.

Opérateurs arithmétiques

Ces opérateurs sont utilisés pour effectuer des opérations arithmétiques. Ils sont au nombre de cinq et requièrent en général des opérandes de type numérique. (sauf `%`).

Opérateur	Opération	Remarque
+	addition	
−	soustraction ou moins unaire	
*	multiplication	
/	division	division entière si les deux opérandes sont de type entier, réelle sinon
%	modulo	seulement pour opérandes de type entier

Tableau 8 : Opérateurs arithmétiques

L'opérateur `%`, appelé modulo, donne le reste de la division entière de deux entiers.

Opérateurs relationnels

Ils permettent de comparer les valeurs des deux opérandes formant l'expression. Le résultat d'une comparaison est de type booléen et vaut

- `true` si le résultat est vrai
- `false` si le résultat est faux

Opérateur	Opération
==	égalité
!=	inégalité
<	plus petit
>	plus grand
<=	plus petit ou égal
>=	plus grand ou égal

Tableau 9 : Opérateurs relationnels

L'expression `5 < 9` donne le résultat `true` (vrai).
L'expression `5 >= 9` donne le résultat `false` (faux).

Opérateurs logiques

Ces opérateurs sont utilisés pour effectuer des opérations logiques sur des opérandes de type booléen.

Opérateur	Opération
!	négation logique
&&	ET logique
||	OU logique

Tableau 10 : Opérateurs logiques

Dans l'expression `e1 && e2`, le résultat vaut `true` si `e1` et `e2` sont tous les 2 vrai. Si `e1` est faux, `e2` n'est pas évalué et le résultat est `faux`.

Dans l'expression `e1 || e2`, le résultat vaut `true` si `e1` ou `e2` est vrai. Si `e1` est vrai, `e2` n'est pas évalué et le résultat est `vrai`.

Incrémentation et décrémentation

L'affectation s'effectue souvent sous forme d'incrémentation ou de décrémentation d'une valeur constante de `1`.

```
j = j+1;
i = i-1;
```

Pour optimiser les programmes, C dispose de deux opérateurs spéciaux : `++` et `--`.

Chacun de ces deux opérateurs peut se placer

- avant l'opérande : il s'agit alors d'opérateur de pré incrémentation, respectivement de pré décrémentation :

  ```
  ++i; // équivalent à i=i+1;
  --j; // équivalent à j=j-1;
  ```

  Le résultat de l'expression est la valeur de l'opérande après application de l'opérateur.
- après l'opérande : il s'agit alors d'opérateur de post incrémentation, respectivement de post décrémentation :

  ```
  i++;
  j--;
  ```
  Le résultat de l'expression est la valeur de l'opérande avant application de l'opérateur.

Comme le nom l'indique, l'action de l'opérateur se fait avant ou après l'évaluation de l'expression.

```
y=x++;
```

Si `x` vaut par exemple `15`, alors `y` prendra la valeur `15` puis `x` sera incrémenté à `16`; le résultat de l'expression est `15`.

```
y=++x;
```

Si `x` vaut `15`, `x` est d'abord incrémenté à `16`, puis `y` reçoit la valeur `16`; le résultat de l'expression est `16`.

Opérateurs de décalage

Ces opérateurs permettent d'effectuer des décalages de bits vers la gauche ou vers la droite pour des entiers.

Opérateur	Opération
<<	décalage à gauche
>>	décalage à droite

Tableau 11 : Opérateurs de décalage

`e1 << e2`

`e1` est décalé de `e2` bits à gauche, avec remplissage de zéros à droite.

`e1 >> e2`

`e1` est décalé de `e2` bits à droite ; si `e1` est non signé ou positif, des zéros sont ajoutés à gauche, sinon cela dépend de l'environnement.

Opérateurs logiques bit-à-bit

Ces opérateurs sont disponibles en C pour effectuer des opérations binaires directement sur les bits de codage pour des opérandes de type entier.

Opérateur	Opération
`&`	ET bit-à-bit
`|`	OU bit-à-bit
`^`	OU exclusif bit-à-bit

Tableau 12 : Opérateurs logiques bit-à-bit

Ordre de précédence des opérateurs

Lorsqu'une expression comporte plusieurs opérateurs, l'évaluation de l'expression se fait selon l'ordre de précédence ou priorité des opérateurs ainsi que des règles d'associativité. Cependant, la priorité est accordée aux expressions entre parenthèses, quels que soient les opérateurs qui les précèdent, ou les suivent.

Précédence	Operateur	Description	Association

1 (la priorité la plus élevée)	`++ --`
	`()`
	`[]`
	`.`
	`->`
	`(type){list}`
		Suffix/postfix increment and decrement
		Function call
		Array subscripting
		Structure and union member access
		Structure and union member access through pointer
		Compound literal(C99)
			Gauche à droite

2	`++ --`
	`+ -`
	`! ~`
	`(type)`
	`*`
	`&`
	`sizeof`
	`_Alignof`
		Prefix increment and decrement[note 1]
		Unary plus and minus
		Logical NOT and bitwise NOT
		Cast
		Indirection (dereference)
		Address-of
		Size-of
		Alignment requirement(C11)
			Droite à gauche

3	`* / %`	Multiplication, division, and remainder	Gauche à droite
4	`+ -`	Addition and subtraction	
5	`<< >>`	Bitwise left shift and right shift	
6	`< <=` `> >=`	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
7	`== !=`	For relational = and ≠ respectively	
8	`&`	Bitwise AND	
9	`^`	Bitwise XOR (exclusive or)	
10	`|`	Bitwise OR (inclusive or)	
11	`&&`	Logical AND	
12	`||`	Logical OR	
13	`?:`	Ternary conditional	Droite à gauche

14	`=`
	`+= -=`
	`*= /= %=`
	`<<= >>=`
	`&= ^= |=`
		Simple assignment
		Assignment by sum and difference
		Assignment by product, quotient, and remainder
		Assignment by bitwise left shift and right shift
		Assignment by bitwise AND, XOR, and OR	

15	`,`	Comma	Gauche à droite

Tableau 13 : Ordre de précédence des opérateurs

Exemple 5.1

```
int main()
{
    int n1=1, n2=20, n;     // declarations
    int cond=0;

    n3 = n2*3 + 5;          // n3=(n2*3)+5, n3 is equal to 65
    n3 -= n2;               // n3 is now equal to 65-20=45
    n2 = n3 + n1++;         // n2 is equal to 45+1=46, and n1 is equal to 2
    cond = n2 == 46;        // as n2 is 2, le result of the comparison is true, so cond is equal to 1

    cond *= 10;             // cond = 1*10 = 10
    n1 = cond%3;            // n1 is equal to 1
                            //(10 modulo 3 = 1 because of integer division of 10 by 3 = 3, it stays 1)
}
```

Figure 4 : Affectations et expressions

Attention

L'ordre d'évaluation des sous-expressions est indéfini. Il dépend du compilateur et ne se fait pas forcément de gauche à droite.

- ```
  int y=2,x=0;
  y=x=x++;//y may have the value 0 !
  ```
- ```
  x = fct1(10) + fct2(20);
  //the calling sequence of fct1 and fct2 is not defined!
  ```

5.2 Conversions de types

Les types arithmétiques peuvent être mélangés dans les expressions et affectations. Lorsqu'une expression contient des opérandes de types différents mais compatibles, il y a conversion implicite.

Les conversions implicites qui préservent les valeurs sont appelées promotions.

Pour les opérandes de type intégral et énumérations, il y a promotion d'entiers, c'est-à-dire que les opérandes de type `bool`, `char`, `short int` ou `enum` sont convertis en `int`.

Pour les nombres réels, l'opérande de taille inférieure est converti automatiquement dans le type de l'opérande de la plus grande taille :

- si un opérande est de type `double`, l'autre opérande devient de type `double` et le résultat est de type `double`
- si un opérande est de type `float`, l'autre devient de type `float` et le résultat est de type `float`, etc.

Ces conversions sont aussi valides dans le cas où l'opérande de taille inférieure est un entier.

Attention

Lors d'une affectation, le type du résultat du membre de droite est converti dans le type du membre de gauche, ce qui peut amener une promotion ou une troncation !

Exemple 5.2

```c
int main()
{
    int integer = 100;
    float real  = 0.0f;

    real = integer + 10.5;      // real is equal to 100.0 + 10.5 -> 110.5
    integer = real;             // truncation : integer = 110

    real = 4/3;                 // Warning :
                                // as 4 and 3 are both integers,
                                // the result of the divison is an integer
                                // -> 1, and real = 1.0 !
}
```

Figure 5 : Conversions implicites

Cet exemple montre qu'il est parfois nécessaire de convertir explicitement une valeur d'un certain type en une valeur d'un autre type (*explicit cast*).

Il existe néanmoins deux autres formes de notation pour la conversion explicite de type, issues du C :

- la notation de coercition (notation traditionnelle), où le nom du type résultat est écrit entre parenthèses;

  ```c
  float x;
  int a=9, b=2;
  x = (float)a/b; // x is 4.5
  ```

- la notation fonctionnelle, où le type du résultat est utilisé comme un appel de fonction (cette notation ne peut pas être utilisée pour les types n'ayant pas un nom simple; cependant son utilisation est préférable pour les exemples complexes).

  ```c
  x = float(a)/b;
  ```

5.3 Fonctions mathématiques

La librairie `<math.h>` met à disposition du programmeur C les fonctions mathématiques les plus courantes.

abs	acos	asin	atan
atan2	atof	cabs	ceil
cos	cosh	exp	fabs
floor	fmod	frexp	hypot
labs	ldexp	log	log10
matherr	modf	poly	pow
pow10	sin	sinh	sqrt
tan	tanh		

Tableau 14 : Fonctions mathématiques

Exemple 5.3

```c
#include <stdio.h>
#include <math.h>

int main()
{
    float x, y;
    double z;
    printf("Base = ");
    scanf("%f",&x);
    printf("Exponent = ");
    scanf("%f",&y);
    z = pow(x,y);
    printf("%f power %f = %f",x,y,z);
}
```

Figure 6 : Utilisation d'une fonction mathématique

6 Instructions de contrôle

C dispose d'instructions de contrôle du flux d'un programme qui traduisent les structures fondamentales de la programmation structurée, à savoir :

- la séquence
- la sélection
- l'itération

Ce chapitre les présente ainsi que les instructions de saut, avec des exemples choisis montrant en général la correspondance entre structogrammes GNS, dont les définitions figurent en annexe A, et traduction C.

6.1 Séquence ou bloc

La séquence d'instructions en C est appelée bloc ou instruction composée. Elle est délimitée par les accolades `{` et `}` qu'on aligne l'une sous l'autre par convention.

```
{
  t = a;      // exchange
  a = b;
  b = t;
}
```

Les blocs peuvent être imbriqués. La déclaration d'un nom dans un bloc le rend accessible dans ce bloc et dans les blocs inclus. Elle peut éventuellement en masquer une autre faite dans le bloc conteneur.

6.2 Sélections

Les instructions de sélection, appelées aussi instructions conditionnelles, permettent de choisir l'exécution d'un flux d'instructions en fonction d'une condition ou valeur d'une expression. Il existe deux sortes d'instructions de sélection en C :

- l'instruction `if`
- l'instruction `switch`

Instruction `if`

Syntaxe

```
if ( expression ) instruction
if ( expression ) instruction1 else instruction2
```

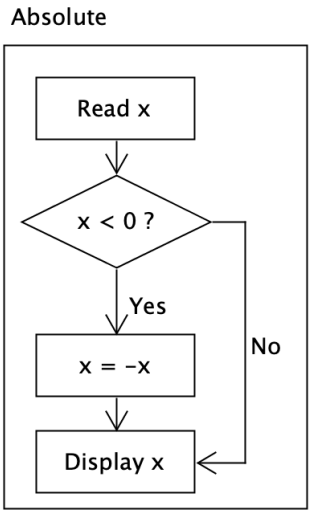
L'expression doit être de type compatible booléen. L'expression est évaluée ; si sa valeur est différente de zéro, donc vraie, alors le programme exécute l'instruction numéro un, sinon s'il y a un `else`, il exécute l'instruction numéro deux. S'il y a plusieurs instructions `if` imbriquées, le `else` est associé au `if` le plus proche.

Les instructions suivant la clause expression et la clause `else` peuvent être des instructions simples, comme une affectation, ou des structures de contrôle. L'instruction

précédant la clause `else` est toujours suivie d'un point-virgule sauf lorsqu'elle se termine par une accolade fermée, par exemple dans le cas d'une séquence.

Ces instructions ne peuvent être des déclarations.

Exemple 6.1

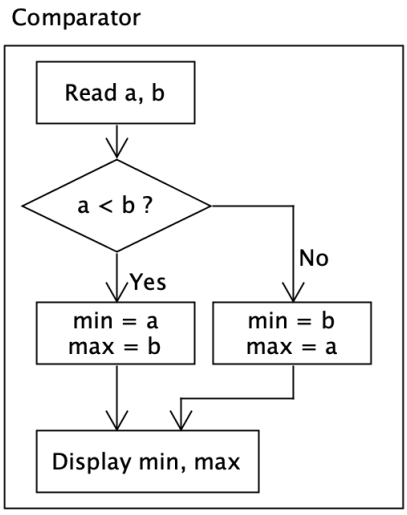



```c
#include <stdio.h>

int main()
{
    int x;
    printf("Enter a positive or negative number ");
    scanf("%i",&x);
    if (x<0) x = -x;
    printf("%i\n",x);
}
```

Figure 7 : Programme qui force un entier x lu au clavier à être positif et l'affiche

Exemple 6.2




```c
#include <stdio.h>

int main()
{
    int a, b, min, max;
    printf("Give a number ");
    scanf("%i",&a);
    printf("Give another number ");
    scanf("%i",&b);
    if (a < b)
    {
        min = a;
        max = b;
    }
    else
    {
        min = b;
        max = a;
    }
    printf("%d is less than %d\n",min,max);
}
```

Figure 8 : Programme qui lit deux entiers, recherche et affiche min et max

Remarque

Il existe en C des expressions conditionnelles de la forme

```
expr1   ?   expr2   :  expr3
```

Cette expression signifie : si `expr1` est vrai, le résultat de l'expression vaut `expr2`, sinon il vaut `expr3`. Elle équivaut à :

```
if (expr1)
{
  expr2;
}
else
{
  expr3;
}
```

Instruction `switch`

Syntaxe

```
switch ( expression ) {
  case constExpression    : instruction;
  ...
  case constExpression    : instruction;
  default                 : instruction;
}
```

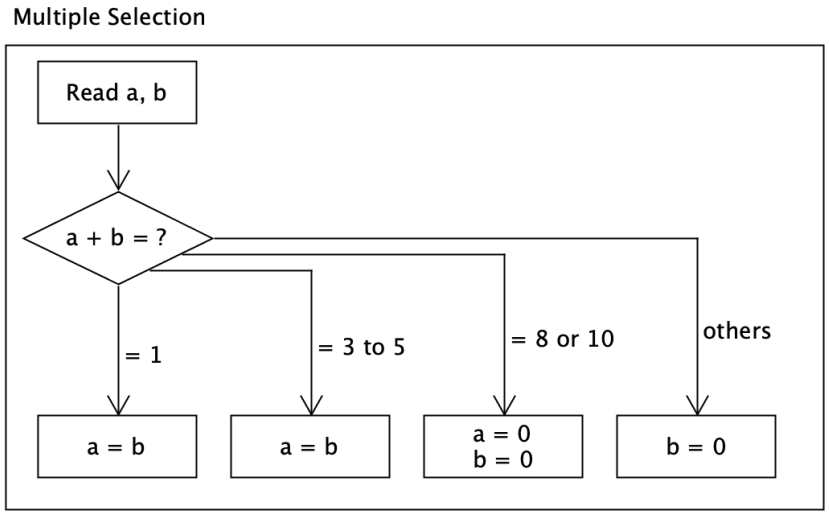
Cette instruction de choix multiple permet de sélectionner une instruction en fonction des valeurs possibles d'une expression.

Les instructions appartenant à un `switch` sont étiquetées par un ou plusieurs labels de choix (`case` ou `default`). L'expression doit être de type entier. Elle est évaluée et comparée avec chaque expression constante de choix. En cas d'égalité, le contrôle passe à l'instruction qui suit le label de choix correspondant. Lorsque la valeur de l'expression ne correspond à aucune constante de choix :

- s'il existe un label `default`, le contrôle est passé à l'instruction étiquetée par `default`
- si `default` n'existe pas, le contenu de l'instruction `switch` n'est pas exécuté

 © HES-SO VS / rim 2023, v1.0

Exemple 6.3




```c
#include <stdio.h>

int main()
{
    int a, b;
    printf("Give a number ");
    scanf("%i",&a);
    printf("Give another number ");
    scanf("%i",&b);

    switch(a+b)
    {
        case 1:
            a = b;
            break;
        case 3:
        case 4:
        case 5:
            b = a;
            break;
        case 8:
        case 10:
            a = 0;
            b = 0;
        default :
            b = 0;
    }
    printf("a=%d,b=%d\n",a,b);
}
```

Figure 9 : Sélection à choix multiples

Attention

Lorsque l'exécution des instructions d'un `case` est terminée, le contrôle ne passe pas à la fin du `switch` mais continue l'exécution du contenu des `case` suivants ! C'est pourquoi, à la fin du contenu de chaque `case`, il faut mettre l'instruction de saut `break` pour provoquer la terminaison de l'instruction `switch`.

6.3 Itérations

Les instructions d'itération sont des boucles permettant de répéter l'exécution d'une instruction.

Il existe trois sortes de boucles en C :

- l'instruction `while`
- l'instruction `do...while`
- l'instruction `for`

Instruction `while`

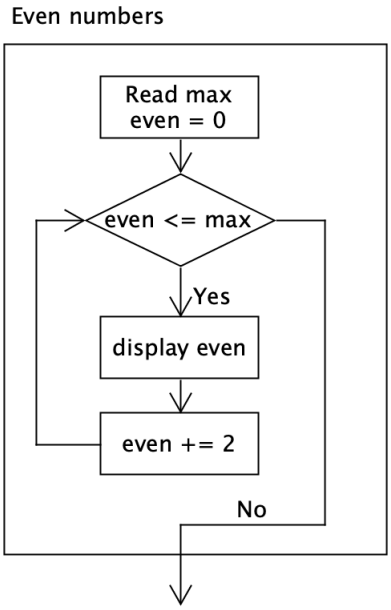
Syntaxe

```
while (expression) instruction
```

Cette instruction correspond à la structure TANTQUE... FAIRE : tant que l'expression est vraie, donc différente de zéro, l'instruction est exécutée.

L'expression de type compatible booléen doit obligatoirement être entre parenthèses. Si sa valeur est nulle au départ, donc fausse, le contenu de la boucle n'est pas exécuté.

Exemple 6.4




```c
#include <stdio.h>

int main()
{
    int max, even = 0;
    printf("Give a number ");
    scanf("%i",&max);
    while(even <= max)
    {
        printf("%d ",even);
        even += 2;
    }
    printf("\n");
}
```

Figure 10 : Affichage des nombres paris de 0 à max (max > 0) avec boucle while

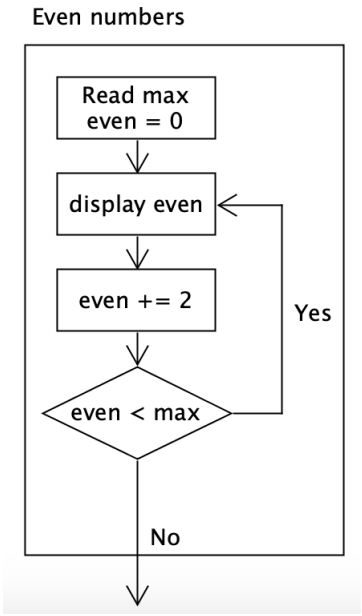
Instruction `do...while`

Syntaxe

```
do instruction while (expression);
```

Cette instruction est utilisée pour traduire la structure itérative classique RÉPÉTER ... JUSQU'À sous une forme légèrement différente, traduite littéralement par RÉPÉTER ... TANTQUE. Le contenu de la boucle est répété tant que l'expression est vraie. Le contenu d'une telle boucle est toujours exécuté au moins une fois puisque le test de l'expression se fait à la fin de la boucle. Si l'instruction ne se termine pas par une accolade fermée, elle doit être suivie d'un point-virgule.

Exemple 6.5




```c
#include <stdio.h>

int main()
{
    int max, even = 0;
    printf("Give a number ");
    scanf("%i",&max);
    do{
        printf("%d ",even);
        even += 2;
    } while(even <= max);
    printf("\n");
}
```

Figure 11 : Affichage des nombres pairs de 0 à max (max > 0) avec boucle `do...while`

Remarque

Il est important de remarquer l'inversion de la condition entre la formulation de l'organigramme GNS et la traduction C.

Instruction `for`

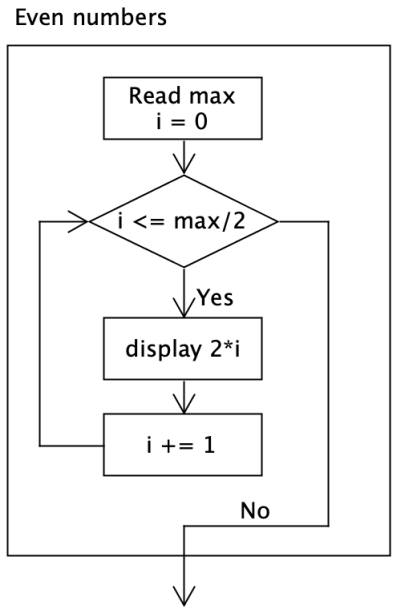
Syntaxe

```
for ( initExpr; testExpr; incrExpr )    instruction
```

L'instruction `for` correspond à la structure FOR ... DO ... . Elle permet d'exécuter de manière répétée une ou plusieurs instructions en comparant la variable de contrôle à une condition d'itération avant chaque début de boucle, puis en modifiant sa valeur par un incrément ou un décrément :

- `initExpr` est l'expression utilisée pour initialiser la variable de contrôle. La variable de contrôle doit être définie hors de l'instruction `for`.
- `testExpr` est l'expression contenant la condition d'itération de la boucle ; cette expression doit être vraie pour que le contenu de la boucle soit exécuté ; elle est testée avant chaque itération ;
- `incrExpr` est une expression définissant l'incrément ou le décrément de la variable de contrôle (la valeur de l'incrément n'est pas limitée à `1`).

Exemple 6.6




```c
#include <stdio.h>

int main()
{
    int max, even = 0;
    printf("Give a number ");
    scanf("%i",&max);

    for (even=0; even<=max; even+=2)
    {
        printf("%d ",even);
    }
    printf("\n");
}
```

Figure 12 : Affichage des nombres pairs de 0 à max (max >0) avec boucle for

Une boucle `for` peut utiliser une variable de type énumération :

Exemple 6.7

```c
#include <stdio.h>

int main()
{
    typedef enum color {blue,white,red,green} color;
    enum color word;
    int i;

    printf("Available colors:\n blue ,white, red, green\n");
    printf("Here are the number of those colors :\n");
    for (word=blue; word <= green; word=(color)(word+1))
    {
        printf("%d ",word);
    }
    printf("\nEnter a number between 0 and 3 : ");
    scanf("%d",&i);

    word = (color)i;
    switch (word) {
        case blue   : printf("You choose blue\n");
                break;
        case white  : printf("You choose white\n");
                break;
        case red  : printf("You choose red\n");
                break;
        case green   : printf("You choose green\n");
                break;
        default     : printf("This color does not exist\n");
    }
}
```

Figure 13 : Utilisation d'énumérations

6.4 Sauts

Les instructions de saut permettent de transférer le contrôle du programme. On en distingue quatre :

- l'instruction `break`
- l'instruction `continue`
- l'instruction `return`
- l'instruction `goto`

Instruction `break`

Elle ne peut apparaître que dans une itération ou dans une instruction `switch`. Elle permet de quitter la boucle en cours ou l'instruction `switch` en provoquant un saut à la fin de la structure de contrôle.

Instruction `continue`

Elle ne peut se trouver que dans une boucle et permet de sauter les instructions qui la suivent en provoquant un retour au début de la structure de contrôle.

```
int i;
for (i=1; i<10; i++)
{
  scanf("&i",&number);
  //...
  if (number<0) continue;   // continue the iteration without
                            // executing the following
                            // instructions
  //...
}
```

Instruction `return`

Elle provoque le retour d'une fonction à l'instruction qui suivait son appel. Elle est suivie d'une expression donnant la valeur du résultat à retourner.

Instruction `goto`

Elle transfert le contrôle à une instruction étiquetée par un label identificateur.

```
startOver:        // label
//...
goto startOver;   // causes the jump to the instruction
                  // who follow the beginning label
```

Remarque

Cette instruction est à proscrire en programmation structurée !

7 Fonctions

L'outil de définition de sous-programmes en C est la fonction. Ce chapitre montre comment construire une fonction, comment gérer le passage des paramètres et l'accès aux données. Il explique également le principe de surcharge de fonction.

7.1 Décomposition algorithmique

L'approche descendante ou *top down* pratiquée en programmation structurée préconise de décomposer un problème en sous-problèmes et ainsi de suite, en identifiant les actions à entreprendre. Cette technique, s'appuyant sur le principe « *diviser pour régner* » permet de réduire la complexité d'un problème en divisant ce dernier en parties qui à leur tour sont affinées. Un programme sera ainsi décomposé en sous-programmes ayant chacun une tâche particulière. Cette décomposition algorithmique caractérise la programmation procédurale.

Exemple : soit le problème de la résolution des équations du second degré. Une première décomposition donne l'organigramme suivant :

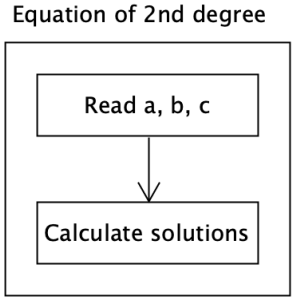


Figure 14 : Résoudre un polynôme de 2éme dégrée

Le sous-problème `calculer solutions` se décompose lui-même en sous problèmes :

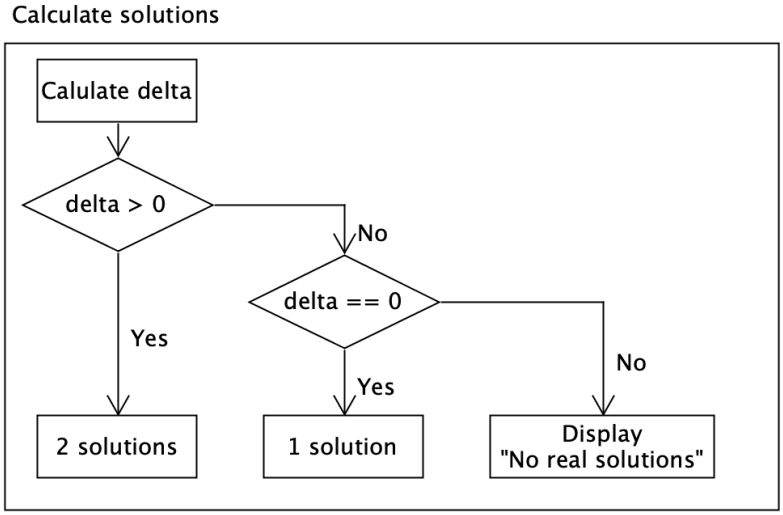


Figure 15 : Calculer le nombre des solutions

Voici la solution du sous-problème `twoSolutions` :

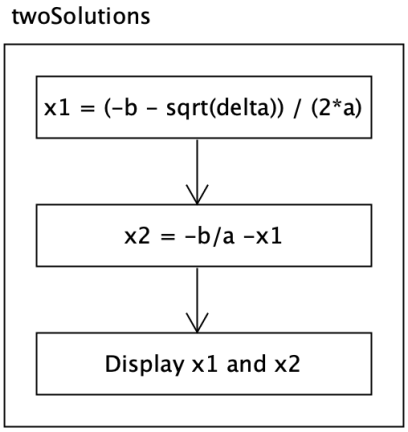


Figure 16 : Calculer les deux solutions réelles

Ainsi, à chaque niveau de décomposition, l'organigramme obtenu sera traduit par un sous-programme. Le langage C n'a qu'une sorte de sous-programme appelé fonction. Rappelons que l'imbrication de fonctions n'est pas possible dans ce langage.

7.2 Déclaration d'une fonction

Toute fonction doit être déclarée avant de pouvoir être utilisée. La déclaration d'une fonction a pour but d'indiquer au compilateur le nom de la fonction, le type du résultat retourné par celle-ci, ainsi que le nombre et le type de ses paramètres ou arguments ; cette déclaration, appelée prototype, introduit le nom de la fonction dans la portée et la rend accessible dans ce contexte.

Syntaxe

```
resultType functionIdentifier(type1 param1, type2 param2,...);
```

`resultType` est le type du résultat retourné par la fonction, `functionIdentifier` est l'identificateur ou nom de la fonction, `param1` est l'identificateur du premier paramètre formel de type `type1` etc.

Une fonction qui ne retourne pas de résultat doit spécifier `void` comme type de résultat. Une fonction de type `void` est équivalente à une procédure en Pascal.

```
float f1(int a, float b);
void f2(char c);
int f3();
```

Remarque

Dans le prototype d'une fonction C, les noms des paramètres formels ne sont pas obligatoires, mais seulement leurs types ; par ailleurs, la liste de paramètres peut être `void`, ce qui indique que le nombre des paramètres est zéro. Si la liste des paramètres est trois points (. . .), le nombre des paramètres peut être quelconques. Ceci demande par contre de au moins spécifier un paramètre avant les trois points.

7.3 Définition d'une fonction

La définition d'une fonction est la description de celle-ci, comprenant l'en-tête et le corps de la fonction.

L'en-tête est identique au prototype de la fonction, avec la liste des paramètres formels et leur type, mais sans le point-virgule final.

Le corps est un bloc d'instructions délimité par les accolades.

Toute fonction qui doit retourner un résultat le fera par l'instruction de saut `return` suivie d'une expression spécifiant la valeur du résultat. L'expression doit être du type du résultat de la fonction. L'exécution de l'instruction `return` provoque la sortie immédiate de la fonction. Il peut ainsi y avoir plusieurs instructions `return` dans une même fonction, mais seule la première rencontrée sera exécutée.

Exemple 7.1

```c
#include <stdio.h>

int max(int,int,int);

int main()
{
    int a,b,c;
    printf("Give a number ");
    scanf("%i",&a);
    printf("Give a number ");
    scanf("%i",&b);
    printf("Give a number ");
    scanf("%i",&c);

    printf("%d\n",max(a,b,c));
}

int max(int a, int b, int c)
{
    int m = (a > b) ? a : b;
    return  (m > c) ? m : c;
}
```

Figure 17 : Définition de la fonction max qui retourne la plus grande valeur parmi les 3

Les définitions de fonctions ne peuvent pas être imbriquées. Lorsqu'un programme contient plusieurs fonctions, celles-ci sont définies les unes à la suite des autres, en général à la fin du programme, c'est-à-dire après la fonction `main`.

Une fonction peut être récursive, c'est à dire qu'elle peut s'appeler elle-même. Dans ce cas, il faut veiller à terminer correctement la suite des appels récursifs afin d'éviter une boucle infinie du programme.

Exemple 7.2

```c
#include <stdio.h>

long int factorial(int);

int main()
{
    int a;
    printf("Give a number ");
    scanf("%i",&a);

    printf("%d\n",factorial(a));
}

long int factorial(int n)
{
    if (n == 0)
        return 1;
    else return n*factorial(n-1);
}
```

Figure 18 : Définition de la fonction factorielle sous forme récursive

7.4 Appel d'une fonction

Une fonction ne peut être appelée que si sa déclaration est accessible à partir de la portée de l'appel. L'appel se fait en indiquant le nom de la fonction, suivi entre parenthèses de la liste des paramètres réels. Les appels de fonction font partie des instructions-expressions.

Lorsqu'une fonction est appelée, chaque argument formel est initialisé avec l'argument réel correspondant, avec vérification du type et, si nécessaire, application des règles de conversion.

```c
int max(int a, int b, int c);  // prototype
int nbr1, nbr2, mx;
//....
mx = max(nbr1,nbr2*10,1000);    // call
```

Attention

L'appel à une fonction n'ayant pas de paramètres se fait en notant le nom de la fonction suivi obligatoirement des parenthèses vides.

Exemple 7.3

```c
#include <stdio.h>

int max(int,int,int);

int main()
{
    int a,b,c;
    printf("Give a number ");
    scanf("%i",&a);
    printf("Give a number ");
    scanf("%i",&b);
    printf("Give a number ");
    scanf("%i",&c);

    printf("%d\n",max(a,b,c));
}

int max(int a, int b, int c)
{
    int m = (a > b) ? a : b;
    return  (m > c) ? m : c;
}
```

Figure 19 : Programme qui lit 3 entiers et affiche la plus grande valeur parmi les 3

7.5 Variables locales et variables globales

Les variables déclarées à l'intérieur d'une fonction sont appelées variables locales. Leur portée ou *scope,* c'est-à-dire la zone où elles sont accessibles, s'étend du point de déclaration jusqu'à la fin du bloc dans lequel la déclaration du nom apparaît.

Les variables globales sont celles qui sont déclarées en dehors d'une fonction ou d'une classe. Leur portée s'étend du point de déclaration jusqu'à la fin du fichier.

Une déclaration de nom dans un bloc peut masquer une déclaration dans un bloc englobant ou un nom global, car un nom peut être redéfini pour désigner une entité différente dans un bloc. A la sortie du bloc, le nom désigne à nouveau l'entité du bloc englobant.

Exemple 7.4

```c
#include <stdio.h>

static int x = 100;          // x global
void f(void);

int main()
{
    printf("case A : %d\n",x++);
    f() ;
    printf("case B : %d\n",x);
}

void f()
{
    int x;                   // x local in f
    x = 1;                   // set local x to 1
    printf("case C : %d\n",x);
    {                        // new block
        int x;               // new local x,
                             // the previous local x is hidden
        x = 3;               // set new local x to 3
        printf("case D : %d\n",x);
    }                        // end of block

    printf("case F : %d\n",x);
    ++x;                     // incrememement local x
                             // (its value is now 2)
    printf("case G : %d\n",x);
}
```

Figure 20 : Exemple du scope

Exemple 7.5

```c
#include <stdio.h>
#include <math.h>

static float a, b, c, delta;        // global var

void calculateSolutions(void);          // prototype

// --------------------------------
// Main programm
// --------------------------------
int main(void)
{
    printf("\nEntrez les 3 coefficients : ");
    scanf("%f",&a);
    scanf("%f",&b);
    scanf("%f",&c);
    calculateSolutions();           // call
}                                   // end of main program

// --------------------------------
// Functions definition
// --------------------------------
void calculateSolutions()
{
    float discriminant();
    void twoSolutions();
    void oneSolution();
    delta = discriminant();
    if (delta > 0) twoSolutions();
    else if (delta == 0) oneSolution();
    else printf("\nThere is no real solutions");
}

float discriminant()
{
    return b*b - 4*a*c;
}

void twoSolutions()
{
    float x1, x2;
    x1 = (-b -sqrt(delta)) / (2*a);
    x2 = -b/a -x1;
    printf("\nThere is 2 solutions : x1 = %f, x2 = %f\n",x1,x2);
}

void oneSolution()
{
    printf("\nOnly one solution : x = %f",(-b/(2*a)));
}
```

Figure 21 : Résolution de l'équation du 2ème degré

7.6 Passage de paramètres

Grâce aux paramètres, un sous-programme peut échanger des informations avec l'extérieur. En C, il existe deux sortes de passages de paramètres :

- le passage par valeur
- le passage par pointeur (référence)

Ces deux sortes de passages sont déterminés par la syntaxe utilisée lors de la définition de la liste de paramètres, dans le prototype et la définition de la fonction. La déclaration d'un paramètre est nommé paramètre formel, la valeur qui est effectivement passée lors de l'appel est nommée paramètre formel.

Passage par valeur

Syntaxe

```
paramType paramIdentifier
```

Ce type de passage concerne les paramètres utilisés par la fonction pour recevoir des informations de l'extérieur. Dans ce cas, lors de l'appel à la fonction, il y a copie de l'argument effectif. Ainsi la fonction travaille sur une copie locale du paramètre actuel transmis, sans affecter l'original.

Passage par pointeur (référence)

Syntaxe

```
paramType * paramIdenfier
```

Le passage par référence est principalement utilisé :

- lorsqu'en plus du résultat, une fonction doit transmettre des informations à l'extérieur
- lorsqu'en paramètre transmis doit être modifié
- lorsque l'on doit transmettre à une fonction un gros objet : on évite ainsi la création d'une copie locale de ce dernier

  Dans ce cas, il est recommandé de déclarer `const` le paramètre formel, pour indiquer que la référence est utilisée seulement pour des raisons d'efficacité et non pour permettre de modifier la valeur de l'objet :

  ```
  void f(type* const someLargeObject);
  ```

Exemple 7.6

```
void f(int val, int* ref)   //val by value,
{                           //ref by pointzer (reference)
    val++;                  //increment the local copy
                            //of first argument
    *ref++;                 //increment the second argument
                            //representing the address of
                            //the effective argument
}


void g()
{
    int i = 1;
    int j = 1;
    f(i,j);                 //after the call, i is equal to 1
                            //and j is equal to 2
                            //note: you can not pass a litteral
                            //as second argument
}
```

Figure 22 : Passage de paramètres par valeur et par référence

7.7 Fonctions `inline`

Chaque appel de fonction ralentit dans une certaine mesure l'exécution d'un programme. Pour les applications sensibles au temps d'exécution, le C offre la possibilité des fonctions compilées en ligne ou `inline`.

Syntaxe

```
inline resultType fctIdenfifier(type1 param1,type2 param2,...)
{
  // ...body of the function
}
```

Le spécificateur `inline` est une demande au compilateur de remplacer chaque appel de la fonction par une copie complète du code de cette fonction. Le compilateur décidera de lui-même s'il y a effectivement lieu de satisfaire cette requête. En général, seules des fonctions très courtes seront compilées en ligne.

7.8 Type du résultat d'une fonction

Le prototype et la définition d'une fonction doit spécifier le type du résultat retourné par celle-ci. Ce type peut appartenir aux types fondamentaux ou aux types dérivés sauf des des tableaux ou des pointeurs de fonction.

Ainsi, par exemple, une fonction peut retourner la valeur d'une variable locale. Elle peut également retourner un pointeur. Dans ce cas, il faudra veiller à ne pas retourner l'adresse d'une variable locale, puisque la place occupée par cette dernière sera libérée à la sortie de la fonction.

8 Pointeurs

Ce chapitre présente les pointeurs, très utilisés en C.

8.1 Pointeurs

L'utilisation de variables de type pointeur permet de travailler sur l'adresse d'une variable (par opposition à son contenu). Un pointeur est en fait une adresse ; cette notion en C n'est pas liée uniquement à l'allocation dynamique de données.

8.2 Déclaration de pointeurs

Une variable de type pointeur est une variable qui contient l'adresse d'un objet. Les pointeurs sont typés, ils sont liés au type de l'objet sur lequel ils pointent.

Syntaxe

```
identificateurType* identificateurPointeur;
```

L'exemple ci-dessous déclare une variable pointeur appelée `pti` qui pointe sur un entier, ainsi que deux pointeurs `f` et `g` sur des réels :

```
int* pti;
float* f, *g;
```

Utilisation de pointeurs

La déclaration d'une variable de type pointeur ne fait que réserver de la place pour ce pointeur, sans l'initialiser. Il faut donc le faire pointer vers un objet en lui affectant l'adresse de l'objet avant de s'en servir.

L'adresse de l'emplacement mémoire d'un objet s'obtient grâce à l'opérateur d'adressage `&` suivi du nom de l'objet.

L'opérateur d'indirection `*` appliqué à un pointeur permet d'obtenir le contenu de l'adresse, donc l'objet pointé lui-même.

Exemple 8.1

```c
#include <stdio.h>

int main(void)
{
    int n1, n2;         //declaration of 2 integers
    int* ptrN;          //declaration of a pointer prtNb
                        //who points on an integer
    n1 = 10;            //nb1 contains the value 10
    ptrN = &n1;         //initialization of the pointer ptrNb who
                        //contains now the address of nb1
    n2 = *ptrN;         //we affect to the content of the object
                        //pointed by ptrNb, it means the value 10
    printf("%d\n",n2);  //display the value 10
}
```

Figure 23 : Opérateurs d'adressage et d'indirection

Il est également possible d'initialiser un pointeur en ne le faisant pointer sur "rien" grâce à la valeur 0 :

```
int *ptr;
ptr=NULL;              // ptr does not point to anything
                       //...
if (ptr==NULL) printf("invalid address\n");
```

Remarque

En C, il est fréquent d'utiliser une macro appelée NULL pour représenter la valeur 0 d'un pointeur. En C, il est préférable d'éviter l'usage de macros. Par conséquent, si l'on veut pouvoir utiliser NULL, il suffit de déclarer :

```
static const void* NULL = (void*)0;
```

Les seules opérations permises sur les pointeurs eux-mêmes sont les affectations, les comparaisons et l'addition ou soustraction de constantes. Ces deux dernières opérations ont la particularité de prendre en compte le type de l'objet pointé :

```
long *pti;    // pti pointer to an integer long
char *ptc;    // ptc pointer to a char

pti++;   // the address pti is incremented by 4 (long = 4 bytes)
ptc++;   // the address ptc is incremented by 1 (char = 1 bytes)
```

Exemple 8.2

```c
#include <stdio.h>

int main()
{
    int x = 3;
    int y;
    int* px;    // pointer to an integer
    int* py;    // pointer to an integer

    px = &x;    // we affect the address of y to the pointer px
    y = *px;    // we affect to y lthe content of the address px
                // it means the value x
    py = &y;    // we affect the address if y to the pointer py

    printf("\nx = %d, &x = %p\npx = %p, *px = %d",x,&x,px,*px);
    printf("\ny = %d, &y = %p\npy = %p, *py = %d\n",y,&y,py,*py);
}
```

Figure 24 : Utilisation de pointeurs

L'exécution de ce programme donne les résultats suivants :

```
x = 3, &x = 0x7ff7bb43497c
px = 0x7ff7bb43497c, *px = 3
y = 3, &y = 0x7ff7bb434978
py = 0x7ff7bb434978, *py = 3
```

Pointeurs et constantes

Le mot-clé `const` peut modifier la déclaration d'un pointeur pour couvrir 3 situations différentes selon sa position :

- déclaration d'un pointeur constant
  `int *const ptr1 = &x;`

  `ptr1` est un pointeur constant sur la variable `x`, cela signifie que `ptr1` ne peut changer de valeur, il contient toujours la même adresse, celle de `x`.
  L'initialisation de `ptr1` est obligatoire lors de sa déclaration.

- déclaration d'un pointeur sur une constante
  `int const* ptr2;`

  Le pointeur ptr2 pointe sur une donnée de type entier considérée comme constante uniquement lors d'opération d'indirection par le biais du pointeur.

- déclaration d'un pointeur constant sur une constante
  `const int *const ptr3;`

Exemple 8.3

```c
#include <stdio.h>

int main()
{
    int x=10, y=20;
    const int a=1;

    int *const ptr1 = &x;       // constant pointer
    int const* ptr2;            // pointer to a constant
    const int *const ptr3=&x;   // constant pointer to constant

    ptr1=&y;                    // error : the content of ptr1 is constant
    *ptr1=50;                   // ok, x becomes equal to 50

    ptr2=&y;                    // ok, initialization of ptr2
    *ptr2=30;                   // error : the content of y when addressed
                                // by ptr2 is considered as constant


    y=30;                       // ok
    ptr3=&y;                    // error : the content of ptr3 is constant
    *ptr3=50;                   // error : the content of x when addressed
                                // by ptr3 is considered as constant
}
```

Figure 25 : Pointeurs et constantes

Remarque

Pour une meilleure compréhension, il est suggéré de lire la déclaration de droite à gauche.

9 Tableaux

Ce chapitre explique comment déclarer et gérer des tableaux de taille fixe à une ou plusieurs dimensions.

9.1 Notion de tableau

Un tableau est un ensemble d'éléments ou objets de même type stockés consécutivement en mémoire, regroupés sous le même identificateur et accessibles individuellement grâce à un indice.

Le type de données tableau appartient aux types dérivés. Les éléments d'un tableau peuvent être de n'importe quel type, fondamental ou dérivé.

Le dessin ci-dessous montre un tableau contenant cinq notes, appelé `note` :

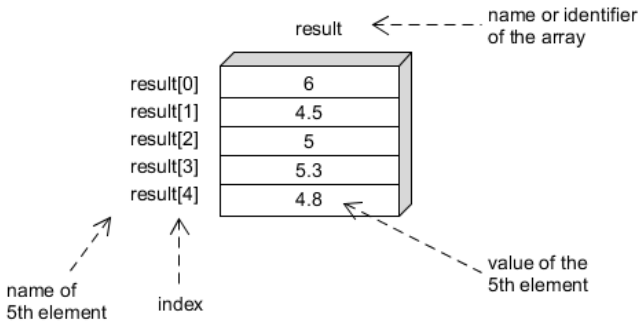


Figure 26 : Représentation d'un tableau

9.2 Tableaux unidimensionnels

Déclaration

Syntaxe

```
identifType identifArray [dimension];
```

`identType` est le type des éléments du tableau, `identifArray` le nom ou identificateur de la variable tableau, et `dimension` une expression constante qui spécifie le nombre d'éléments du tableau.

```
double grades[5];
```

déclare le tableau `note` correspondant au dessin de la page précédente; ceci réserve en mémoire un espace pour cinq réels consécutifs.

```
int integers[20];
```

déclare un tableau `entier` de vingt nombres entiers. Si par exemple, selon le système utilisé, un nombre entier est codé sur 2 octets, alors le tableau occupera une place de `20x2` octets consécutifs en mémoire.

```
char line[80];
```

déclare un tableau `ligne` de quatre-vingts caractères.

Initialisation

Comme pour toute variable C, un tableau est initialisable lors de sa déclaration :

```
int digit[10] = {0,1,2,3,4,5,6,7,8,9};
```

Il est également possible de n'initialiser que le début d'un tableau :

```
char alphabet[26] = {'A','B','C'};
```

Lorsque l'initialisation est faite à la déclaration, la spécification de la taille n'est pas nécessaire ; le tableau prend automatiquement comme taille le nombre de valeurs spécifiées.

```
int number[] = {4,8,3,2,7};
```

La taille du tableau `number` est automatiquement fixée à cinq éléments.

Accès aux éléments du tableau

Chaque élément d'un tableau est accessible individuellement grâce à un indice, entre crochets `[]`, indiquant sa position dans le tableau ; cet indice démarre toujours à zéro.

```
int i, digit[10];
for (i=0; i<10; i++)
{
  digit[i] = i;
}
```

Ceci a le même effet que la déclaration et l'initialisation vue plus haut.

```
int digit[10] = {0,1,2,3,4,5,6,7,8,9};
```

Attention

En C, les éléments d'un tableau de dimension `n` sont numérotés de `0` à `n-1`. Ainsi dans l'exemple du tableau `grades`, `grades[0]` désigne la première note du tableau, et `grades[4]` la cinquième et dernière note. Il n'y a pas de contrôle de débordement du tableau en C !

9.3 Tableaux multidimensionnels

Les tableaux à plusieurs dimensions sont représentés comme des tableaux de tableaux.

Déclaration

Syntaxe

```
identifType identifierArray [ dim1][dim2] [....][dimn];
```

Voici 2 exemples:

```
int matrix[4][3];
float cube [10][10][10];
```

La déclaration du tableau `matrix` réserve en mémoire un espace pour quatre tableaux consécutifs de trois entiers chacun.

Initialisation

Un tableau multidimensionnel est également initialisable lors de sa déclaration.

```
int matrix[4][3]= {{0,1,2},{3,4,5},{6,7,8},{9,10,11}};
```

Accès aux éléments du tableau

Chaque élément du tableau est accessible par sa position dans le tableau. Il faut donner autant d'indices qu'il y a de dimensions.

```
int matrix[4][3];
for (int i=0; i<4; i++)          // same effect
{
  for (int j=0; j<3; j++; j++)   // as the initialization
  {
    matrix[i][j] = 3*i+j;        // of the previous sample
  }
}
```

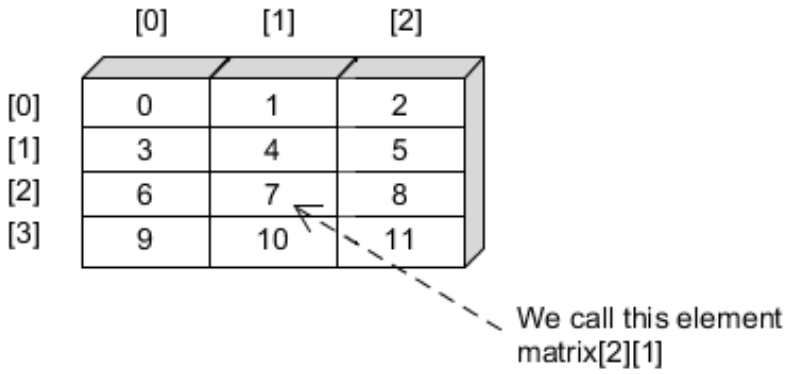


Figure 27 : Le tableau **`matrix`**

Remarque

Afin d'éviter des erreurs, il est recommandé d'utiliser de préférence les conteneurs `vector`, `list`, `string` etc. de la bibliothèque standard.

9.4 Tableaux et pointeurs

En C, les tableaux sont étroitement liés aux pointeurs. En effet, lors de la déclaration d'un tableau, par exemple :

```
int arr[10];
```

si `arr[i-1]` désigne le ième élément du tableau `arr`, l'identificateur ou nom du tableau `arr` représente en fait l'adresse du premier élément du tableau. Cette adresse est considérée comme un pointeur constant. Ainsi, la notation `arr` est équivalente à `&arr[0];` la notation `arr+1` est équivalente à `&arr[1]` etc.

La manipulation de tableaux à l'aide de pointeurs est fréquente et plus rapide à l'exécution. Il suffit de déclarer un pointeur, lui donner l'adresse du premier élément du tableau, puis incrémenter cette adresse pour parcourir le tableau. L'incrémentation du pointeur tient automatiquement compte de la taille de l'élément.

Exemple 9.1

```c
int main(void)
{
    int i, arr[10], *ptrArr;

    ptrArr = arr;   //ptrArr receives the address of the first element

    for(i=0; i<10; i++) *(ptrArr + i) = 222;
                    //each element of the array receives the value 222
                    //equivalent to
                    //for(i=0; i<10; i++) *(arr + i) = 222;
                    //or
                    //for(i=0; i<10; i++) arr[i] = 222;
}
```

Figure 28 : Initialisation d'un tableau à l'aide d'un pointeur

9.5 Paramètres tableaux

Concernant le passage de paramètres, les tableaux diffèrent des autres types et ne peuvent être transmis par valeur.

En effet, lors de la transmission d'un tableau comme paramètre en C, bien que l'on utilise la syntaxe du passage par valeur, en fait c'est un pointeur sur le premier élément du tableau qui est transmis, c'est-à-dire l'adresse du tableau. Par conséquent, la taille du tableau n'est pas disponible dans la fonction appelée. C'est pourquoi très souvent la taille est également passée comme paramètre.

A l'intérieur de la fonction, toute modification d'un élément du tableau transmis agira donc sur l'original.

Les deux exemples qui suivent montrent l'initialisation d'un tableau à l'aide d'une fonction et l'affichage de la somme de ses éléments.

Exemple 9.2

```c
#include <stdio.h>
const int MAX=10;

int main()
{
    double arr[MAX];
    void fillArray(double array[MAX]);
    double sum(double array[MAX]);
    fillArray(arr);
    printf("The sum of the elements is = %f\n",sum(arr));
}

void fillArray(double array[MAX])   // MAX doesn't have any use
{
    int i;
    for (i=0; i<MAX; i++)
    {
        printf("value: ");
        scanf("%lf",&array[i]);
    }
}

double sum(double array[MAX])
{
    double total=0;
    int i;
    for (i=0; i<MAX; i++) total += array[i];
    return total;
}
```

Figure 29 : Tableau version A

Dans cette version, l'information `MAX` n'est connue des fonctions `fillArray` et `sum` que parce que c'est une constante globale ; même si elle figure dans la liste de paramètres, ce n'est pas un paramètre. C'est pourquoi la version suivante est préférable.

Exemple 9.3

```c
#include <stdio.h>
const int MAX=10;

int main(void)
{
    double arr[MAX];
    void fillArray(double array[], int size);
    double sum(double array[], int size);
    fillArray(arr,MAX);
    printf("The sum of the elements is = %f\n",sum(arr,MAX));
}

void fillArray(double array[], int size)
{
    int i;
    for (i=0; i<size; i++)
    {
        printf("value: ");
        scanf("%lf",&array[i]);
    }
}

double sum(double array[], int size)
{
    double total=0;
    int i;
    for (i=0; i<size; i++) total += array[i];
    return total;
}
```

Figure 30 : Tableau version B

Pour la transmission de tableaux multidimensionnels, le problème est plus complexe. Ainsi par exemple pour une matrice, transmettre la valeur de la 2e dimension selon le modèle précédent ne fonctionne pas. La solution consiste à passer comme paramètre un pointeur sur le tableau et non le tableau.

Exemple 9.4

```c
#include <stdio.h>
const int ROW=4;
const int COL=3;

int main()
{
    double matrix[ROW][COL] = {{1,2,3},{4,5,6},{7,8,9},{10,11,12}};
   double sum(double* addrArray, int dim1, int dim2);
   printf("The sum of the elements = %lf",sum(matrix,ROW,COL));
}

double sum( double* addrArray, int dim1, int dim2)
{
    double total=0;
    int i;
    int nbElem=dim1 * dim2;
    for (i=0; i<nbElem; i++) total += *(addrArray + i);
    return total;
}
```

Figure 31 : **matrix** en paramètre

9.6 Opérateur `sizeof`

Le C fournit un opérateur unaire appelé `sizeof` qui retourne la taille en octets de son opérande. Cette taille dépend de l'environnement. L'opérande peut être une expression, un objet, un tableau, etc. Lorsqu'il s'agit d'un tableau, le résultat est donc le nombre total d'octets du tableau.

```c
int array[] = {0,1,2,3,4};
printf("%d",sizeof(array));     // will display 20
```

Cette technique peut être utile pour éviter des erreurs dues à une initialisation incomplète d'un tableau.

Attention

L'utilisation de `sizeof` dans une fonction recevant un tableau comme paramètre ne permet pas de connaître la taille du tableau transmis. Son utilisation dans ce contexte donne comme résultat la taille de l'adresse du tableau, c'est-à-dire la taille des pointeurs sur ce système !

10 Chaînes de caractères

A la différence d'autres langages, le C n'offre pas de type de base pour la manipulation de chaînes de caractères ou `string`. Ce chapitre explique la méthode héritée du langage C de représenter les chaînes à l'aide de tableaux de caractères terminés par le caractère 0 (`NULL, \0`) et de les gérer au moyen des outils de la librairie `<string.h>`.

10.1 Nature d'une chaîne de caractères

Une chaîne de caractères est codée dans un tableau dont chaque élément contient un caractère de la chaîne, ou plus exactement sa valeur numérique propre au jeu de caractères utilisé (par exemple ASCII) ; le tableau doit contenir après le dernier caractère de la chaîne un élément supplémentaire marquant la fin de chaîne, qui a la valeur `0` et peut être identifiée par la séquence `'\0'`.

L'utilisation d'une chaîne de caractères peut se faire selon les cas à travers :

- l'introduction d'une constante littérale de type chaîne de caractères, entre guillemets, dans le texte du programme
- la déclaration et utilisation d'une variable chaîne de caractères, de taille fixe
- la déclaration et utilisation d'un pointeur sur une chaîne de caractères

Littéraux chaînes de caractères

Une constante chaîne de caractères est tapée directement dans le programme, entre guillemets, en général lors de déclaration de constantes ou dans des ordres d'affichage. Une telle chaîne est stockée comme une donnée statique ; le nombre d'éléments du tableau est égal au nombre de caractères de la chaîne plus un (pour la marque de fin de chaîne qui est ajoutée automatiquement).

Lorsqu'une telle constante figure dans une expression, elle est traitée comme adresse du premier caractère de la chaîne.

```
printf("A string");
```

Cette chaîne occupe en mémoire neuf emplacements :

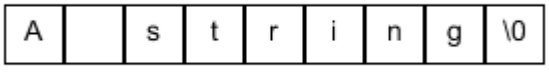


Figure 32 : La chaîne de caractères **"A string"** en mémoire

Variables chaînes de caractères

Déclarer une variable chaîne de caractères consiste à déclarer un tableau de caractères de taille fixe.

```
char text[12];
```

L'initialisation d'une variable chaîne lors de sa déclaration peut se faire par :

- initialisation habituelle de tableaux
  ```
  char text[12] = {'A',' ','s','t','r','i','n','g','\0'};
  ```
- initialisation par copie d'une chaîne constante
  ```
  char text[12] = "A string";
  ```

Voici le contenu du tableau `text` pour ces deux exemples :

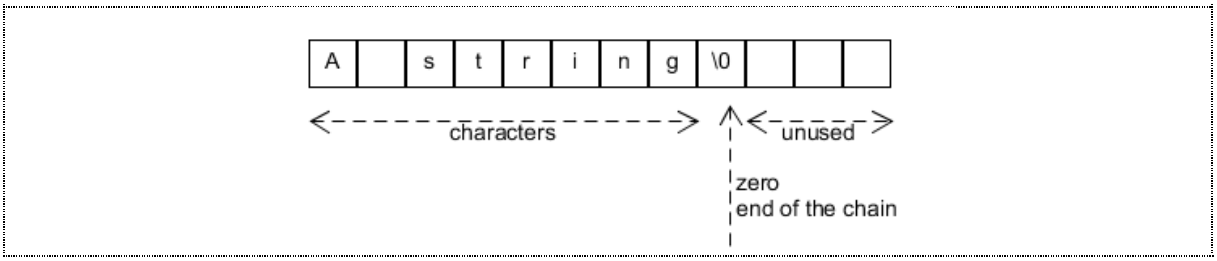


Figure 33 : La chaîne de caractères **`text`**

Si la dimension du tableau n'est pas spécifiée, le tableau aura une taille égale au nombre de caractères de la chaîne d'initialisation plus un.

```
char title[] = "A string";
```

Le tableau `title` contient :

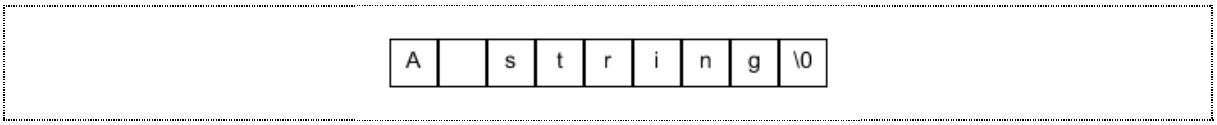


Figure 34 : La chaîne de caractères **`title`**

Chaque caractère du tableau est accessible en donnant le nom du tableau suivi d'un indice entre crochets :

```
title[2] = 'S'; // now title is "A String"
```

Remarque

Lorsque l'on désire stocker un maximum de `n` caractères dans un tableau, il faut déclarer ce tableau avec une dimension de `n+1`, puisque la marque de fin de chaîne occupe un élément.

Attention

Il n'est pas possible d'affecter une nouvelle chaîne de caractères à une variable chaîne en utilisant l'opérateur d'affectation =. Il faut recourir à une fonction de bibliothèque, par exemple `strcpy` :

```
text = "Another string";          // wrong!
strcpy(text,"Another string");    // ok
```

Pointeurs sur chaînes de caractères

L'utilisation de pointeurs sur des chaînes de caractères est très courante. Il suffit de déclarer un pointeur sur caractère et de lui donner l'adresse de la chaîne, par initialisation ou affectation.

```
char* ptr = "A string";
                  // ptr is initialized with the address of
                  // the constant string "A string"

ptr = "Another string";
                  // ptr obtains the address of
                  // the constant string "Another string"
```

Pour empêcher la modification de l'adresse contenue dans un pointeur, il suffit de déclarer le pointeur en `const` :

```
const char* constPtr = "A string";
```

10.2 Opérations sur les chaînes

Ce point décrit les principales opérations sur des variables chaînes de caractères ou pointeurs sur chaînes de caractères.

Soit les déclarations suivantes :

```
char name[20];
char text[] = "A string";
const char* ptr = "Another string";
```

Lecture d'une chaîne

Pour lire une chaîne de caractères entrés au clavier, on peut utiliser :

- la fonction `gets` librairie `<stdio.h>`
  La chaîne va recevoir tous les caractères saisis jusqu'à la fin de la ligne.
  Si le nombre est supérieur à la taille de la chaîne, celle-ci est agrandie.
  `gets(name);`
- la fonction `scanf` librairie `<stdio.h>`
  (fonctionnement identique à `gets`)

  ```
  scanf("%s", name);
  ```

10.3 Affichage d'une chaîne

Pour afficher une chaîne à l'écran, on peut utiliser :

- la fonction `puts` librairie `<stdio.h>`
  fait un retour automatique à la ligne:

```
puts(nom);
puts(ptr);
```

- la fonction `printf` librairie `<stdio.h>`

```
printf("%s",nom);
printf("\n%s\n",text);
printf("%s",ptr);
```

Copie de chaînes

Puisque l'opérateur d'affectation n'est pas défini pour les tableaux, l'affectation d'une nouvelle chaîne de caractères à une variable se fait à l'aide de la fonction de copie `strcpy` de la librairie `<string.h>`.

```
char name1[20], name2[20];
char tmp[20];
                        // ... Exchange of name1 and name2
strcpy(tmp,name1);      // copy the content of name1 in tmp
strcpy(name1,name2);
strcpy(name2,tmp);
```

Comparaison de chaînes

Les fonctions `strcmp(s1,s2)` et `strcmpi(s1,s2)` de la librairie `<cstring>` permettent de comparer le contenu de deux variables chaînes de caractères `s1` et `s2`.

(`strcmpi` ignore la différence entre majuscules et minuscules pour les lettres non accentuées)

Si

```
s1 < s2    :    la fonction retourne une valeur entière négative
s1  == s2  :    la fonction retourne la valeur 0
s1  >  s2  :    la fonction retourne une valeur entière positive
```

```
if(strcmpi(name1,name2) > 0)
{
    //... exchange them
}
```

Longueur d'une chaîne

La fonction `strlen` permet d'obtenir la longueur d'une chaîne de caractères transmise comme paramètre.

Elle retourne le nombre de caractères effectifs, sans compter la marque de fin de chaîne.

```
char title[20] = "Lesson C";
printf("%d",strlen(title));    // displays 8
```

Fonctions de la libraire **<string.h>**

La librairie `<string.h>` offre plusieurs fonctions prédéfinies pour le traitement de chaînes de caractères, dont voici une liste des plus courantes :

```
memccpy         memchr          memcmp          memcpy
memicmp         memmove         memset          movedata
movmem          setmem          stpcpy          strcat
strchr          strcmp          strcmpi         strcpy
strcspn         strdup          strerror        strlen
strlwr          strncat         strncmp         strncmpi
strncpy         strnset         strpbrk         strrchr
strrev          strset          strspn          strstr
strtod          strtok          strupr          strxfrm
```

11 Structures et unions

Les structures permettent de regrouper des données de types différents. Ce chapitre explique l'utilisation du type structure et de ses variations, les champs de bits et unions, qui appartiennent tous les trois à la catégorie des types dérivés du langage C.

11.1 Structures

Le type de données `struct` est l'équivalent du type enregistrement que l'on trouve dans beaucoup de langages de programmation. Un `struct` permet de regrouper plusieurs données, de types différents, qui qualifient un même objet.

Déclaration de structures

Pour pouvoir manipuler une donnée de type structure, il faut construire d'abord un nouveau type de données, auquel on donne un nom, puis déclarer une variable de ce type.

Les différents éléments composant la structure sont appelés les membres de la structure.

Syntaxe

```
struct identifier {
    type1  member1;
    type2  member2;
    ...
    typeN  memberN;
};
```

`struct` est un mot réservé ; `identifier` est le nom choisi pour ce type de données.

```
typedef struct SheetA {        // declaration of type SheetA
    char name[25];             // declaration of struct members
    int age;
    float size, weight;
} SheetA;


SheetA client;      // client is a variable of type SheetA
```

Remarque

En C, il est obligatoire d'ajouter le mot-clé `struct` devant la déclaration d'une variable de type structure, si la structure n'a pas été "typedefé" :

```
struct SheetA {             // declaration of type SheetA
     char name[25];         // declaration of struct members
     int age;
     float size, weight;
};

struct SheetA client;
```

Les membres d'une variable de type structure peuvent être initialisés lors de la déclaration de la variable.

```
struct SheetA client = {"Dupont", 25, 180, 85};
```

Dans le cas de structures imbriquées, l'initialisation se fait de la même manière, en rajoutant des accolades autour des valeurs du membre de type structure.

```
struct Date {int day, month, year; };
struct SheetB {
  char name[25];
  struct Date birthDate;
  float size, weight;
};
struct SheetB clientX = {"Durand",{3,2,1976}, 190, 86};
struct SheetB  clientZ = {}; //everything is 0;
```

Une variable de type structure peut être déclarée sans nommer le type structure.

```
struct {
      float x,y;
} point;            //point is a variable of type structure
```

11.2 Accès aux membres d'une structure

Il convient de distinguer deux cas pour l'accès aux membres d'une structure :

- Avec une variable de type structure, l'accès se fait au moyen de l'opérateur point `.` :
  On écrit le nom de la variable suivi du point suivi du nom du membre.
  ```
  struct SheetB clientX;
  clientX.size = 190;
  clientX.birthData.year = 1976;
  ```
- Avec un pointeur sur une structure, l'accès se fait au moyen de l'opérateur d'indirection `->` :
  On écrit le nom du pointeur suivi de `->` suivi du nom du membre.
  ```
  struct SheetB* ptrClient;
  ptrClient=&clientX;
  ptrClient->size = 190;      // same as
                              // (*ptrClient).size = 190
  ptrClient->birthDate.year = 1976;
  ```

Utilisation des structures

L'affectation est la seule opération autorisée entre deux variables de type structure, à condition que ces deux variables soient exactement du même type.

```
struct SheetB clientX={"Jean",{1,1,1970},175,68}, clientY;
clientY = clientX;
```

L'affectation membre à membre est bien sûr toujours possible, en respectant la compatibilité de types.

```
clientY.size = clientX.size;// ok
clientY.name = clientX.name;  // !false ! because name is an array
                              // whose address is constant
                              // we should write :
strcpy(clientY.name,clientX.name);
```

Les objets de type structure peuvent être transmis comme arguments et renvoyés comme résultat de fonctions. Toutes les autres opérations telles que comparaisons, entrées-sorties, etc., ne peuvent se faire qu'à partir des membres si leur type le permet.

```
printf("Size of client Y ?");
scanf("%f",&(clientY.size));
if (clientY.size > clientX.size) {
  printf("The client Y is taller");
}
```

Les structures sont très utilisées pour la construction de structures de données telles que listes ou arbres. Dans ce cas, elles contiennent en général des membres de type pointeur.

```
struct Element {
    char info[20];
    element* previous;
    element* next;
};
struct Element list[10];
```

Une fonction peut retourner un résultat de type structure, ou un pointeur sur une structure.

```
struct  MyStruct {  ...  };        // declaration of type mystruct
struct MyStruct func1(void);       // return a structure
struct MyStruct* func2(void);      // return a pointer to a structure
```

Une structure peut être passée comme paramètre à une fonction.

```
void func3(struct MyStruct s);    // by value
void func4(struct MyStruct* s);   // by pointer (reference)
```

Remarque

Selon le matériel utilisé, la taille-mémoire d'un objet de type structure peut être plus grande que la somme de la taille de chacun de ses membres. Cette valeur peut être obtenue en appliquant l'opérateur `sizeof` au type `struct` ou à la variable `struct`.

11.3 Champs de bits

L'octet est le plus petit espace-mémoire adressable pour une donnée de type fondamental. Parfois, certaines données n'occupent en fait que quelques bits. Pour ne pas perdre la place inoccupée, le type `struct` permet de travailler sur des groupes de bits d'un mot-machine grâce à la définition de champs de bits.

Déclaration de champs de bits

Dans une structure, chaque membre est défini comme un champ en précisant à la fin de sa déclaration le nombre de bits qu'il occupe.

Un champ doit être d'un type intégral (`int`, `unsigned int`, `bool`, `char` ou `unsigned char`) ou d'un type énumération.

Syntaxe

```
struct identifier {
     type1  member1 : nbrbits;
     type2  member2 : nbrbits;
     ...
     typeN  memberN : nbrbits;
};
```

Le nombre de bits doit être une valeur entre un et 16 ou 32, selon la taille des mots-mémoire du système ; des champs sans nom sont autorisés ; en général, mais cela dépend de l'environnement, les champs sont stockés dans l'ordre du bit de poids le plus faible au bit de poids le plus fort.

```
struct MyStruct{
  unsigned i  : 2;
  unsigned j  : 5;
  int         : 4;
  int k       : 1;
  unsigned m  : 4;
};

struct MyStruct word = {3,20};// init. of the 2 firsts fields
```

Voici une représentation interne possible de la variable `word`, sur 16 bits :

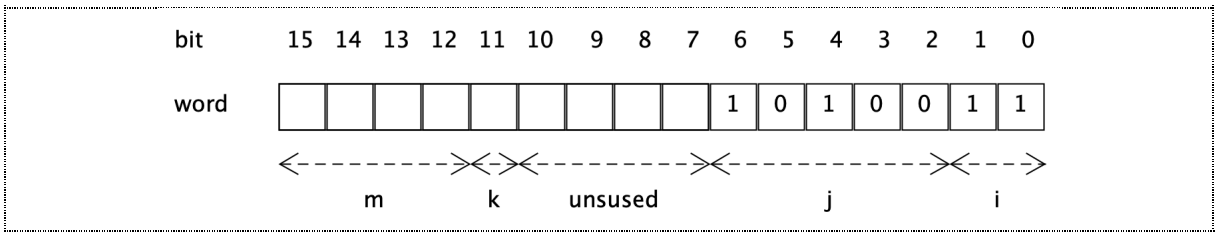


Figure 35 : Représentation de la structure `MyStruct` en mémoire

Utilisation de champs de bits

Une variable champs de bits est utilisée comme une structure. L'accès aux différents champs se fait de la même manière que l'accès aux membres, mais il n'est pas possible de prendre l'adresse d'un champ.

Les opérations courantes sur les entiers ne sont pas applicables à une variable champs de bits. Lorsque la valeur que l'on veut stocker dans un champ est trop grande par rapport au nombre de bits réservés, la valeur est tronquée, les bits de poids fort sont perdus.

Remarque

Les applications comportant des champs de bits sont rarement portables. D'autre part, utiliser des champs n'économise pas forcément de la mémoire : en effet, bien qu'il y ait un gain de place-mémoire pour le stockage des données, la taille du code nécessaire à la manipulation des champs va croître ; le temps total d'exécution va également augmenter.

11.4 Unions

Les unions permettent de stocker des objets de types différents dans un même espace-mémoire.

Déclaration d'une union

Syntaxe

```
union identifier {
     type1  member1;
     type2  member2;
     ...
     typen  membern;
};
```

Une variable de type `union` n'aura qu'un seul membre, choisi dans la liste figurant dans la déclaration du type.

```
union MyUnion{
     int i;
     char c;
};
union MyUnion mu;  // the mu variable can contain, as needed
                   // either an integer or a character
```

La taille d'une variable de type `union` est fixe ; elle correspond à la place nécessaire pour stocker le membre le plus grand.

Utilisation d'unions

L'utilisation d'une variable union est similaire aux structures :

```
union MyUnion mu;
mu.i = 10;              // mu contains the integer 10
printf("%5d",mu.i);
mu.c = 'A';             // mu now contains a character
printf("%c",mu.c);
printf("%d",mu.i);      // this will print the ASCII value of 'A'
```

Cependant, une variable union ne peut être initialisée lors de sa déclaration qu'à travers le premier membre.

Remarque

Le compilateur ne peut en général pas savoir quel membre est utilisé à un moment donné ; il ne peut donc y avoir de vérification de type correcte. C'est au programmeur d'assurer une utilisation cohérente de l'union.

Exemple 11.1

```c
#include <stdio.h>

enum Sorts{integer, character, real};

struct Record{
    enum Sorts nature;
    union {             // anonymous union (without any words)
        int i;
        char c;
        float f;
    };
};

void display(struct Record val);

int main()
{
    struct Record valueI = {integer,{10}};
    struct Record valueC = {character,{'A'}};
    struct Record valueR = {real,{5.55f}};
    display(valueI);
    display(valueC);
    display(valueR);
}

void display(struct Record val)
{
    switch (val.nature) {
        case integer:
            printf("Integer %i\n",val.i);
            break;
        case character:
            printf("Character %c\n",val.c);
            break;
        case real:
            printf("Real %f\n",val.f);
            break;
    };
}
```

Figure 36 : Structure avec union

12 Allocation dynamique

Ce chapitre montre comment créer et gérer des données dynamiques, et présente en exemple les tableaux et listes dynamiques.

12.1 Allocation dynamique de mémoire

Les objets déclarés dans un programme se voient attribués un emplacement en mémoire pour y stocker leurs valeurs. Ils occupent en général la partie de mémoire appelée pile ou `stack`. La taille de ces objets doit être connue à la compilation, et c'est le compilateur qui détermine leur emplacement mémoire. Leur durée de vie est influencée par leur classe de stockage. Ces objets continuent à occuper de l'espace-mémoire jusqu'à ce qu'ils perdent leur validité, même si l'on n'en a plus besoin.

Il est parfois utile de pouvoir créer des objets au cours de l'exécution d'un programme, au fur et à mesure des besoins du moment, et de pouvoir détruire ces objets lorsqu'ils sont devenus inutiles. Le langage C offre cette possibilité grâce à l'allocation dynamique de mémoire, dans une partie de mémoire appelée communément le tas ou `heap`, ou mémoire libre.

Les objets créés dynamiquement sont des éléments dont le nombre ou la taille n'est pas connu à la compilation : par exemple des éléments de tableaux, des nœuds d'arbres ou éléments de listes chaînées. La durée de vie de ces objets est gérée par le programmeur.

12.2 Allocation dynamique en C

Dans le langage C, il faut utiliser les fonctions standards `malloc` et `free` de la librairie `<stdlib.h>`. La fonction `malloc` exige comme paramètre la taille en octets de l'objet à créer en octets.

```
char* str;
str = (char*) malloc(10);          // allocates a memory location
                                   // for the chain str
strcpy(str, "Hello");              // copy "Hello" in the chain str
printf("String is %s\n", str);     // displays : String is Hello
free(str);                         // released the memory location
```

12.3 Exemples

L'exemple suivant montre la création d'un tableau dynamique à une dimension, appelé `arr`, dont la `size` est entrée au clavier à l'exécution du programme.

Exemple 12.1

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int size, i;
    int* arr;

    printf("\nEnter the size of the array: ");
    scanf("%d",&size);

    //allocation of the array (integer size):

    arr = (int*)malloc(sizeof(int)*size);

    //Check if the allocation was successfully done :

    if (arr == nullptr) printf("Memory allocation error\n");
    else {
        // Work with the array, for example :

        for (i=0; i<size; i++)
        {
            arr[i] = i;
        }

        //desallocation of the array at the end of the work :
        free(arr);
    }
}
```

Figure 37 : Tableau dynamique à une dimension

L'exemple qui suit qui suit montre la création d'un tableau dynamique à deux dimensions, dont le nombre de lignes et de colonnes est déterminé à l'exécution du programme.

Pour allouer un tableau à plusieurs dimensions, il faut indiquer la taille de chaque dimension entre crochets ; cette taille peut être une variable pour la première dimension, mais doit être une constante pour les dimensions suivantes. Or ici, le nombre de lignes et de colonnes sont des variables dont la valeur est fixée en cours d'exécution. Il n'est donc pas possible de créer directement un tableau à deux dimensions. On alloue tout d'abord un tableau de pointeurs, avec un pointeur par ligne, puis pour chaque pointeur de ligne on alloue un tableau dont le nombre d'éléments est le nombre de colonnes.

Exemple 12.2

```c
#include <stdio.h>
#include <stdlib.h>

int **t;  // pointer to an integer array
int i, j, rows, columns;

int main(void)
{
    printf("\n Enter the number of rows and columns : ");
    scanf("%d", &rows);
    scanf("%d", &columns);

    // Allocate of an array of pointers
    // each cell of this array will point
    // to a row of the 2 dimensional array

    t = (int**) malloc(sizeof(int*)*rows);

    // for each row, allocate of the number of columnss
    for (i=0; i<rows; i++)
    {
        t[i] = (int*) malloc(sizeof(int)*columns);
    }
    // Initialization of the array
    for (i=0; i<rows; i++)
    {
        for (j=0; j<columns; j++)
        {
            printf("t[%d,%d] :",i,j);
            scanf("%d",&t[i][j]);
        }
    }
    //display the array
    for (i=0; i<rows; i++)
    {
        for (j=0; j<columns; j++)
        {
            printf("t[%d,%d] = %d ",i,j,t[i][j]);
        }
        printf("\n");
    }
    // Destruction
    for (i=0; i<rows; i++)
    {
        free(t[i]);
    }
    free(t);
}
```

Figure 38 : Tableau dynamique à 2 dimensions

L'exemple qui suit montre la création et la gestion d'une liste linéaire composée de noms. Une liste linéaire est une structure de données constituée d'éléments chaînés, où chaque élément possède des informations qui lui sont propres ainsi qu'un pointeur sur l'élément suivant. En général, les éléments sont créés dynamiquement.

Dans cet exemple, le programme lit un nom, crée dynamiquement l'élément avec ce nom et l'ajoute simplement à la fin de la liste, sans tri. Ainsi, si les noms entrés étaient dans l'ordre Sam Sheepdog, Ralph Wolf et Marvin the Martian, on obtiendrait la liste suivante :

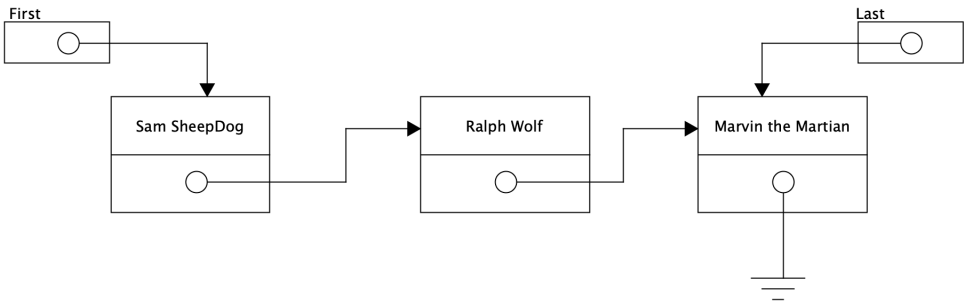


Figure 39 : Liste chaînée

Exemple 12.3

```c
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 20
typedef struct tElement{
    char name[MAX+1];
    struct tElement* next;
} tElement;

tElement* first = 0, * last = 0 ;

void addNode(void);
void displayList(void);

int main(void)
{
    char choice;
    printf("Add a name (Y/N) ? ");
    fflush(stdin);
    scanf("%c",&choice);

    while (toupper(choice) != 'N')
    {
        addNode() ;

        printf("Add a name (Y/N) ? ");
        fflush(stdin);
        scanf("%c",&choice);
    }
    displayList();
}
```

```c
void addNode(void)
{
    tElement* elem;
    elem = (tElement*) malloc(sizeof(tElement));

    printf("Enter a name : ");
    fflush(stdin);
    scanf("%s",elem->name);
    elem->next = 0;

    if (last == 0)
    { first = elem; last = elem ;}
    else
    { last->next = elem; last = elem;}
}

void displayList(void)
{
    tElement* p;
    p = first;

    if (p == 0)
        printf("The list is empty ...\n");
    else
    {
        printf("\nContent of the list : \n");
        while (p  != 0)
        {
            printf("%s\n",p->name);
            p = p->next;
        }
    }
}
```

Figure 40 : Programme pour stocker des noms dans une liste chaînée

13 Classes de stockage

En C, un programme est composé d'un ou de plusieurs fichiers. Dans le cas d'un programme multifichiers, chaque fichier est constitué d'une suite de déclarations et peut être compilé séparément. Les fichiers compilés sont ensuite assemblés par le linker pour obtenir un programme exécutable.

Tout objet d'un programme mono ou multifichiers doit être déclaré et défini. Lorsque l'objet est créé, il est stocké dans un emplacement-mémoire et devient accessible selon des règles dépendant de l'endroit de la déclaration, de la façon dont l'objet a été déclaré et de sa durée de vie.

Après avoir introduit les notions de portée, durée de vie et emplacement-mémoire, ce chapitre explique l'utilisation d'espaces de noms et la spécification de classes de stockage.

13.1 Définitions

Portée

La portée d'un objet définit la région du programme dans laquelle l'objet peut être utilisé. En C, il existe plusieurs types de portée : locale et globale.

Portée locale
Un objet déclaré dans un bloc, c'est-à-dire à l'intérieur d'une fonction ou dans un bloc d'instructions délimité par des accolades, est local à ce bloc. Il n'est donc accessible qu'à l'intérieur de ce bloc dans lequel il est défini ainsi que dans les blocs imbriqués.

Les noms des arguments formels d'une fonction ont une portée locale limitée au corps de cette fonction.

Portée globale
Un objet déclaré dans un fichier à l'extérieur de toute fonction, bloc ou classe a une portée de fichier. Un tel objet est appelé objet global. Il est accessible à partir de son point de déclaration dans tout le fichier et peut être utilisé par toutes les fonctions à l'intérieur de ce fichier sans devoir être passé comme paramètre.

Dans le cas d'un programme multifichiers, un objet global défini dans un fichier peut être accessible dans les autres fichiers à ces conditions :

- l'objet doit être déclaré et défini comme global dans un seul fichier
- les autres fichiers qui veulent utiliser ce même objet global doivent le redéclarer avec le même type et nom, avec une portée locale ou globale, en précédant la déclaration du spécificateur de stockage `extern`

Remarque

Dès leur définition, les fonctions ont une portée globale puisqu'elles ne peuvent être imbriquées.

Durée de vie

La durée de vie signifie la durée pendant laquelle un objet existe dans un emplacement mémoire. En général, on distingue les deux durées de vie suivantes :

- durée de vie automatique (objets automatiques)
- durée de vie statique (objets statiques)

Objets automatiques
Les objets avec une durée de vie automatique n'existent que pendant un appel de fonction ou pendant l'exécution d'un bloc d'instructions pour lequel ils sont définis. C'est la situation par défaut des objets locaux.

Lorsque l'exécution du bloc commence, la définition de l'objet provoque la réservation d'un emplacement-mémoire pour stocker l'objet. Lorsque l'exécution du bloc est terminée, l'emplacement- mémoire de l'objet est libéré. Un objet automatique a donc une existence dynamique : il n'est créé qu'à l'exécution du bloc et disparaît après.

Objets statiques
Les objets avec une durée de vie statique restent en mémoire pendant toute la durée d'exécution d'un programme. Ces objets possèdent une place fixe dans la mémoire. Les objets globaux sont des objets statiques.

Un objet local peut être défini comme statique : dans ce cas, bien que n'étant accessible que dans le bloc où il est défini, l'objet ne meurt pas à la sortie du bloc; à chaque nouvelle entrée dans le bloc, il retrouve la valeur qu'il avait lors de la dernière sortie de ce bloc.

Emplacement mémoire

Une autre caractéristique des objets est l'emplacement-mémoire. Il existe deux endroits différents pour le stockage des objets :

- la mémoire
- les registres

Objets dans la mémoire principale
En général, les objets sont stockés dans la mémoire. La zone de mémoire réservée au stockage des objets est décomposée en trois parties :

- le segment-données ou *data-segment*, qui contiendra les objets statiques; l'occupation du segment-données est fixe pendant l'exécution du programme puisqu'il contient les objets globaux
- la pile ou *stack*, qui contiendra les objets automatiques. L'occupation de la pile varie pendant l'exécution d'un programme puisque les objets locaux par défaut "meurent" à la sortie du bloc
- le tas ou *heap*, réservé pour les objets dynamiques

Objets dans les registres du processeur

Le stockage des variables dans les registres du processeur est une technique à laquelle on recourt lorsque les variables sont utilisées souvent ou qu'une très grande vitesse de traitement est nécessaire. Bien entendu, cela n'est possible que pour certains types de variables et si des registres sont disponibles. En plus, cette technique n'est pas soutenue par tous les compilateurs C.

13.2 Classes de stockage

Lors de la déclaration d'un objet, il est possible d'influencer sa portée, sa durée de vie ou son emplacement-mémoire à l'aide d'un des spécificateurs de classes de stockage ci-dessous qui sont des mots-clés :

- `auto`
- `static`
- `extern`
- `register`

Le mot-clé spécifiant la classe de stockage se place au début de la déclaration de l'objet, avant son type.

Spécificateur `extern`

Le spécificateur `extern` est utilisé dans le cas d'un programme multifichiers, lorsqu'un fichier utilisateur veut pouvoir accéder à un objet global défini dans un autre fichier.

L'objet global est déclaré et défini dans un seul fichier, de la façon habituelle. Les autres fichiers utilisateurs de cet objet global doivent le redéclarer à l'aide du mot-clé `extern`. Cette déclaration peut se faire selon les besoins dans une portée locale ou de fichier.

Un même objet peut être déclaré plusieurs fois mais défini une seule fois. Une définition est une déclaration sans le mot `extern`, avec éventuellement une initialisation. C'est lors de la définition de l'objet qu'il y a réservation d'emplacement mémoire. Une déclaration `extern` ne fait qu'une référence à cet emplacement. Si on essaie d'initialiser une variable lors de sa déclaration `extern`, au lieu de sa définition, on obtient un message d'erreur.

Remarque

Une variable globale n'est pas externe par défaut ; c'est l'autre fichier utilisateur qui doit la déclarer `extern` ; cependant, une fonction est par défaut `extern` !

Le spécificateur `auto`

Le spécificateur `auto` permet de définir un objet automatique qui sera stocké sur la pile. Ce mot-clé ne peut être employé qu'à l'intérieur de fonctions.

Le mot-clé `auto` est optionnel : si aucun spécificateur n'est mentionné dans la déclaration d'une variable avec portée locale, celle-ci est par défaut automatique.

Le spécificateur `static`

Le spécificateur `static` permet de définir des objets statiques. Ceux-ci sont alors stockés dans un emplacement fixe, le segment-données de la mémoire, et existent pendant toute l'exécution du programme. Par défaut, les objets globaux sont des objets statiques, accessibles dans tout le fichier.

Le spécificateur `static` s'utilise dans deux situations :

- Devant la déclaration d'une variable locale :
  celle-ci devient alors un objet statique. Si la déclaration contient une valeur d'initialisation, la variable ne sera initialisée que lors de la première entrée dans le bloc. Ensuite, à chaque nouvelle exécution du bloc, la variable retrouvera sa valeur précédente.

  Il faut remarquer que même si une variable locale statique persiste pendant l'exécution du programme, elle n'est cependant accessible que dans sa portée ! Une variable locale statique est ainsi un moyen de stocker de façon permanente des données à l'usage exclusif d'une seule fonction.
- Devant la déclaration d'une variable globale ou une définition de fonction :
  la portée de la variable ou de la fonction devient alors limitée au fichier.
  Ceci permet de rendre accessible l'objet dans tout le fichier mais de le cacher aux autres fichiers. On évite ainsi des conflits éventuels entre noms identiques figurant dans plusieurs fichiers.

Spécificateur `register`

Le spécificateur `register` est utilisé dans une déclaration de variable pour demander au compilateur de stocker celle-ci dans un registre du processeur, en vue d'une optimisation lors d'accès fréquents. Dans le cadre des compilateurs actuels modernes, cette spécification est devenue superflue.

13.3 Exemples

Exemple 13.1

```c
#include <stdio.h>

int value1, value2;     // global variables

void exchange(void);

int main(void)
{
    value1 = 1;
    value2 = 2;
    printf("Before : %d, %d\n",value1,value2);
    exchange();
    printf("After  : %d, %d\n",value1,value2);
}

void exchange()
{
    int temp;           // local variables
    temp = value1;
    value1 = value2;
    value2 = temp;
}
```

Figure 41 : Variables globales et locales dans un programme monofichier

Explication

Dans cet exemple, les deux variables `value1` et `value` sont définies à l'extérieur de toute fonction, elles sont donc globales. Elles ont une portée de fichier et sont accessibles dans tout le fichier. Ce sont des variables statiques, avec un emplacement fixe dans le segment-données. Par contre, la variable `temp` déclarée dans la fonction `exchange` a une portée locale. C'est une variable automatique qui n'existera sur la pile que pendant l'exécution de la fonction `exchange`. La fonction `exchange` est un objet global ; elle pourrait ainsi être appelée par d'autres fonctions s'il y en avait.

Exemple 13.2

```
// ------------------------------------------------------
// File MODULE1.C
// ------------------------------------------------------
#include <stdio.h>

float length, width, area;      // Definition of
                                // 3 global variables

void reading(void);             // prototype of the function

int main(void)
{
    reading();
    area = length*width;
    printf("\nArea = %f\n",area);
}


// ------------------------------------------------------
// File MODULE2.C
// ------------------------------------------------------
#include <stdio.h>

extern float length;    // Declaration of 2 external global
extern float width; // var defined in the previous file

void reading()          // Definition of the function reading
{
    printf("Enter length of the rectangle: ");
    scanf("%f",&length);
    printf("Enter width of the rectangle: ");
    scanf("%f",&width);
}
```

Figure 42 : Variables globales dans un programme multifichier

Explication

Dans cet exemple, les variables `length`, `width` et `area` sont définies comme variables globales dans le premier fichier `MODULE1.C`. Pour accéder aux variables `length` et `width`, le deuxième fichier `MODULE2.C` doit les déclarer externes. Elles sont alors accessibles dans tout le fichier `MODULE2.C`. On aurait pu dans cet exemple les déclarer externes uniquement dans la fonction `reading`, pour limiter leur portée.

La fonction `reading` est définie dans le deuxième fichier. Comme une fonction est par définition un objet global, externe, le premier fichier peut utiliser cette fonction en déclarant son prototype sans spécification externe. La variable `area` est globale, mais utilisée seulement dans le premier fichier. Elle pourrait ainsi être déclarée à l'intérieur de la fonction `main`, en tant que variable locale.

Exemple 13.3

A

```c
#include <stdio.h>

void sub(void);
int main(void)
{
    int val;
    val = 1;
    printf("%d\n",val);
    sub();
    printf("%d\n",val);
}

void sub(void)
{
    int val;
    val = 2;
    printf("%d\n",val);
}
```

Exécution
1
2
1

B

```c
#include <stdio.h>

void sub();

int main()
{
    int val;
    val = 1;
    printf("%d\n",val);
    sub();
    printf("%d\n",val);
}

void sub()
{
    val = 2;
    printf("%d\n",val);
}
```

Exécution
`Error : undefined symbol`

Ce programme ne peut pas être compilé

C

```c
#include <stdio.h>

int val;
void sub();

int main()
{
    val = 1;
    printf("%d\n",val);
    sub();
    printf("%d\n",val);
}

void sub()
{
    val = 2;
    printf("%d\n",val);
}
```

Exécution
1
2
2

Figure 43 : Exemple de portées

Explication

Dans le programme `a`, la fonction `main` déclare une variable locale appelée `val`. La fonction `sub` déclare une autre variable locale appelée aussi `val`. Bien que portant le même nom, ces deux variables ne sont accessibles que dans la portée locale où elles sont définies.

Dans le programme `b`, la variable `val` est déclaré avec une portée locale dans la fonction `main`. Elle n'est donc pas accessible dans la fonction `sub`. C'est pourquoi la compilation de ce programme donnera une erreur.

Dans le programme `c`, la variable `val` est déclaré globale. Elle est donc accessible à l'intérieur des fonctions `main` et `sub`.

Exemple 13.4

```c
// ------------------------------------------------------
// Average of 3 numbers
// ------------------------------------------------------
#include <stdio.h>

float getVal(void);                 // Prototypes
void  averageVal(float inVal);

int main(void)
{
    int i;
    for (i=1; i<4; i++)         // 3 numbers
        averageVal(getVal());
}
float getVal(void)
{
    float val ;
    printf("Enter a number ");
    scanf("%f",&val);
    return(val);
}
void averageVal(float inVal)
{
    static float  sum  = 0.0;
    static int    quantity = 0;
    float mVal;
    sum  += inVal;
    ++quantity;
    mVal = sum / quantity;
    printf("Sum        : %f\n", sum);
    printf("Quantity   : %d\n", quantity);
    printf("Average    : %f\n", mVal);
}
```

Result of the program execution if the the values 1,2, and 3 are entered:

```
Enter a number 1
  Sum        : 1
  Quantity   : 1
  Average    : 1
Enter a number 2
  Sum        : 3
  Quantity   : 2
  Average    : 1.5
Enter a number 3
  Sum        : 6
  Quantity   : 3
  Average    : 2
```

Figure 44 : Variables statiques dans un programme monofichier

Explication

Cet exemple montre le calcul simple de la valeur moyenne de trois nombres. Dans la fonction `averageVal`, deux variables statiques de type `float` et `int` sont déclarées. Elles ont une durée de vie statique mais une portée locale. Bien que la fonction `averageVal` soit appelée trois fois et quittée trois fois, les valeurs des variables statiques `sum` et `quantity` sont conservées, car elles ne sont pas stockées sur la pile mais dans le segment-données. La variable `mVal` est par contre une variable automatique. Sa valeur n'est pas conservée lorsqu'on quitte la fonction. Il n'est pas nécessaire de sauvegarder cette valeur, car il suffit de la recalculer à chaque fois. Les variables `sum` et `quantity` ne peuvent être appelées qu'à l'intérieur de la fonction `averageVal` puisqu'elles ont une portée locale.

Exemple 13.5

```
// ------------------------------------------------------
// Fichier MODULE1.C
// ------------------------------------------------------
void calculate(void);

int main(void)
{
    for(int i=1; i<6; i++)
        calculate();
}
```

```
// ------------------------------------------------------
// Fichier MODULE2.C
// ------------------------------------------------------
#include <stdio.h>

static int quantity = 0;

float getVal(void);
void  calculate(void);
void  simpleVal(float in);
void  squareVal(float in);

void calculate(void)
{
    float val;
    val = getVal();
    simpleVal(val);
    squareVal(val);
}

float getVal()
{
    float val;
    printf("Enter a number ");
    scanf("%f",&val);
    ++quantity;
    printf("Number of values     : %d\n",quantity);
    return(val);
}

void simpleVal(float inVal)
{
    static float  sumVal  = 0;
    float  mVal;
    sumVal  += inVal;
    mVal = sumVal / quantity;
    printf("Sum des values     : %f\n", sumVal);
    printf("Average            : %f\n", mVal);
}

void squareVal(float inVal)
{
    static float  sumCar  = 0;
    float  mVal;
    sumCar += inVal*inVal;
    mVal = sumCar / quantity;
    printf("Sum of the squares     : %f\n", sumCar);
    printf("Average og the squares : %f\n", mVal);
}
```

Figure 45 : Variables statiques dans un programme multifichiers

Explication

Dans cet exemple, la variable statique globale `quantity` peut être utilisée partout dans le fichier `MODULE2.C` ; toutes les fonctions de ce fichier peuvent avoir accès à cette variable et la modifier. Les variables statiques locales `sumVal` et `sumSquare` ne sont accessibles respectivement qu'à l'intérieur des fonctions `simpleVal` et `squareVal`.

Les trois variables statiques conservent leur valeur pendant toute l'exécution du programme.

Exemple 13.6

```c
// ------------------------------------------------------
// Comparison of the execution time
// ------------------------------------------------------

#include <stdio.h>
#include <time.h>

void autoDemo(void);
void registerDemo(void);

int main(void)
{
    autoDemo();
    registerDemo();
}

void autoDemo(void)
{
    time_t start, stop;
    int i=0;                    //stack variables
    int n=0;

    start = time(NULL) ;

    for (n=0; n<1000; n++)
        for (i=0; i<30000; i++);

    stop  = time(NULL);
    printf("Execution time (memory ) : %d seconds\n",difftime(stop,start));
}

void registerDemo()
{
    time_t start , stop;
    register int i=0;           // registers vriables
    register int n=0;

    start = time(NULL) ;
    for(n=0; n<1000; n++)
        for(i=0; i<30000; i++);

    stop  = time(NULL);
    printf("Execution time (register) : %d seconds\n",difftime(stop,start));
}
```

Figure 46 : Variables registres

Explication

Cet exemple affiche le temps d'exécution de deux boucles imbriquées, dans les fonctions `autoDemo` et `registerDemo`. Dans la fonction `autoDemo`, les variables de comptage `i` et `n` sont déclarées en tant que variables entières `auto` sauvegardées sur la pile. Dans la fonction `registerDemo` cependant, les variables de comptage `i` et `n` sont déclarées en tant que variables `register`. Si les boucles sont répétées assez longtemps, on observera que cette fonction s'exécute plus rapidement.

14 Directives de prétraitement

Une mise en œuvre de C contient un préprocesseur capable de substitution de macros, de compilation conditionnelle et d'inclusion de fichiers. Les ordres donnés au préprocesseur, appelés directives de prétraitement, ont une syntaxe indépendante du langage et sont liés aux différentes implémentations. Ce sont des outils permettant d'avoir une meilleure vue d'ensemble du programme ou de grands projets, ainsi qu'une réduction du code du programme à ce qui est effectivement nécessaire.

Ce chapitre présente les principales directives, sans entrer dans tous les détails.

14.1 Forme d'une directive

Une directive commence toujours par le signe `#` suivi d'un mot-clé.

Une directive peut apparaître n'importe où dans le programme ; elle a un effet indépendant des règles de portée de C et qui dure en principe jusqu'à la fin de l'unité de traduction, une unité de traduction étant le fichier résultat du prétraitement.

Les directives de prétraitement sont surtout utilisées pour :

- définir des constantes
- définir des macros
- supprimer des définitions de constantes et de macros
- inclure des fichiers
- la compilation conditionnelle
- le contrôle des lignes du programme
- les messages d'erreur pendant la compilation
- l'adaptation aux spécificités de la machine

 © HES-SO VS / rim 2023, v1.0

14.2 `#define` (définition)

Définition de constantes

Syntaxe

`#define identifier symbolChain`

Cette directive provoque le remplacement par le préprocesseur de toutes les occurrences suivantes de l'`identifier` par la séquence constituant la `symbolChain`.

Les constantes définies ainsi n'ont donc pas besoin de place en mémoire. Ces constantes sont globales par rapport au fichier.

```
#define TITLE "Here is a long text"
#define LENGTH 80
#define WIDTH (LENGTH + 20)
```

Si l'on écrit dans le programme :

```
var = WIDTH * 20;
```

Cette séquence après l'étape du prétraitement se présente comme suit :

```
var = (80 + 20) * 20;     // Result = 2000
```

Attention

Dans cet exemple, il ne faut pas oublier de mettre la définition de `length` entre parenthèses afin d'éviter la situation suivante :

```
var = 80 + 20 * 20;       // Result = 480
```

Remarque

En C, il est conseillé d'utiliser `const` au lieu de `#define` pour déclarer les constantes du programme. Les noms déclarés avec `#define` ne sont pas typés, ni limités en portée ; ceux déclarés avec `const` sont typés et suivent les règles de portée de C. L'information de type peut aider le compilateur C à détecter les erreurs. De plus, la limitation de la portée des noms apporte plus de clarté et empêche les conflits de dénomination.

On utilisera `#define` pour déclarer des noms qui ne sont pas des valeurs (par exemple les noms de classes) et chaque fois qu'une substitution de texte est nécessaire.

Définition de macros

Syntaxe

```
#define identifier  (parameters list) symbolChain
```

Les macros définies par `#define` sont très utiles. Le préprocesseur substitue chaque occurrence de la macro par le texte de sa définition. Cela permet par exemple d'éviter de définir une fonction puis de l'appeler à plusieurs reprises, engendrant ainsi tout le mécanisme lié à l'appel. Les macros améliorent la lisibilité des programmes et sont globales.

La liste de paramètres entre parenthèses est optionnelle. Si elle est présente, elle est constituée d'identificateurs séparés par des virgules.

```
#define MAX (a,b) ((a)>(b)  ? (a):(b))
#define cube(X)   (X * X * X)
```

Si l'on écrit dans le programme :

```
x = 3;                  // x is different from X
y = cube(x);
z = cube(x+1);          // This result is false !
```

Après le passage dans le préprocesseur, on aura :

```
x = 3;
y = x * x * x;
z = x+1 * x+1 * x+1;    // we are expecting
                        //  (x+1)*(x+1)*(x+1)
```

Quelques macros sont prédéfinies dans la norme *ANSI*. Elles commencent et finissent toujours par deux tirets soulignés. Elles n'ont pas de paramètres et ne doivent pas être utilisées en tant qu'identificateurs de nouvelles macros.

`__DATE__`	date du fichier de programme courant
`__FILE__`	nom du fichier de programme courant
`__LINE__`	numéro de la ligne courante dans le fichier de programme courant
`__STDC__`	C courant compatible avec ANSI-C si __STDC__ = 1
`__TIME__`	heure du fichier de programme courant

Tableau 15 : Principales macros prédéfinies

Chaque version C possède en plus une liste de constantes et macros particulières prédéfinies.

Remarque

La directive de prétraitement `#define` définit des constantes ou des macros qui pendant le passage dans le préprocesseur, juste avant la compilation effective, sont substituées dans le code source.

Ces directives ne sont pas intelligentes, c'est-à-dire qu'elles ne se comportent pas comme des fonctions. C'est pourquoi il est très important d'en tenir compte lorsque l'on appelle une macro.

14.3 `#undef` (suppression de définition)

Syntaxe

```
#undef  identifier
```

La directive de prétraitement `#undef` permet de supprimer la définition d'une constante ou d'une macro en indiquant son nom.

Si `#undef` n'est pas utilisé, la définition reste active jusqu'à la fin de l'unité de traduction. Lorsque `#undef` est utilisé avec un nom qui n'a pas été défini, il est ignoré.

14.4 `#include` (inclusion de fichier)

Syntaxe

```
#include <file name>  or  #include "file name"
```

La directive `#include` provoque le remplacement de cette ligne par le contenu entier du fichier spécifié. Elle permet ainsi d'inclure des fichiers en-têtes ou *header files*, contenant des déclarations et définitions de constantes et macros, ayant le suffixe `.h` en général.

Le nom de fichier doit mentionner le suffixe et respecter la syntaxe du système d'exploitation. Il est entouré des signes `<  >` s'il s'agit d'un fichier en-tête standard C ; s'il s'agit d'un fichier utilisateur, le nom de fichier, précédé éventuellement du chemin d'accès, est entouré de guillemets ".

L'inclusion avec `#include` est possiblement récursive, c'est-à-dire qu'un fichier en-tête peut également comprendre des directives `#include`.

```
#include <stdio.h>
#include "mydef.h"
```

14.5 Compilation conditionnelle

Le préprocesseur permet la compilation conditionnelle, c'est-à-dire la possibilité de ne faire traduire qu'une partie du code source en fonction de certaines conditions.

Clause `if`

Syntaxe

```
#if constantExpression
   //...
  #endif

or
  #ifdef identifier
   //...
  #endif

or
  #ifndef identifier
   //...
  #endif
```

Si la `constantExpression` est vraie, donc différente de 0 (cas `#if`), ou si l'identificateur mentionné a bien été défini (cas `#ifdef`) ou n'a pas été défini (cas `#ifndef`), alors les instructions ou directives placées dans cette structure seront compilées.

Il est possible d'y ajouter une ou plusieurs clauses `elseif` suivies chacune du texte à compiler si la `constantExpression` est vraie.

Clause `elif`

Syntaxe

```
#elif constantExpression
```

La directive `#ifdef` est une contraction de la directive `#if` où la `constantExpression` consiste en l'opérateur unaire `defined` suivi d'un identificateur. Dans ce cas, l'expression est vraie si l'identificateur mentionné a été défini par une directive `#define`; il en est de même pour `#ifndef` équivalent à `#if not defined`.

Dès qu'une `constantExpression` vraie est rencontrée, les textes des autres clauses sont ignorés.

Il est possible de mentionner le texte à compiler dans le cas où aucune `constantExpression` n'a été rencontrée, à la suite de la clause "sinon".

Clause `else`

Syntaxe

```
#else
```

Le fait d'exclure de la compilation certaines parties de code est très répandu dans les programmes C. Le programmeur a ainsi la possibilité de regrouper plusieurs fichiers dans une même application. Cette application ne contiendra cependant que les fichiers effectivement utilisés. Grâce à cette technique, des programmes exécutables peuvent ainsi être réduits à leur grandeur optimale.

```
#ifdef CREDIT    // equivalent to #if defined CREDIT
  credit();
#elif defined DEBIT
  debit();
#else
  cout <<"Error";
#endif
```

C méthode peut aussi être utilisée pour la définition conditionnelle de constantes.

```
#if LANGUAGE == 1
  #define ERROR   "Fehlermeldung"
#elif LANGUAGE == 2
  #define ERROR   "Errormessage"
#elif LANGUAGE == 3
  #define ERROR   "Message d'erreur"
#else
  #define ERROR "???"
#endif
```

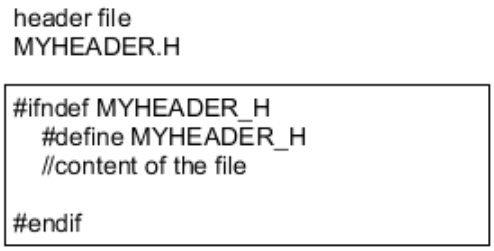
Une application fréquente est l'inclusion de différentes structures de débogage pendant le développement du programme, lesquelles seront ensuite exclues du code du programme.

```
#define NDEBUG
                          // ... and in the code of the program :

#ifdef NDEBUG
     cout << someVar;     // display the variable for checking
#endif
```

Remarque

En général, tous les fichiers en-tête sont construits sur ce modèle :



Cela permet de contrôler et éviter ainsi plusieurs inclusions d'un même fichier.

14.6 `#line` (contrôle des lignes)

Syntaxe

```
#line  constant  "file name"
```

Cette directive de prétraitement positionne la macro prédéfinie `__LINE__` en fixant la valeur de la constante comme nouveau numéro de la ligne qui suit. Si un nom de fichier est spécifié, `__FILE__` contiendra ce nouveau nom.

Ainsi, si une erreur apparaît, le message d'erreur sera donné avec ce numéro de ligne ou avec ce nom de fichier.

```
#line 151 "copy.c"
```

Avec cette directive, les constantes internes `__LINE__` et `__FILE__` prennent respectivement les valeurs `151` et `"copy.c"`.

Cette directive sert avant tout à la détection flexible d'erreurs.

14.7 `#error` (contrôle du message d'erreur)

Syntaxe

```
#error  symbolChain
```

Cette directive de prétraitement est utilisée pour inclure des messages d'erreur émis pendant la compilation.

```
#ifndef errorCheck
  #error Error test not included
#endif
```

Cette méthode permet par exemple de vérifier que l'on a bien éliminé la définition de la constante `NDEBUG`, vue précédemment.

```
#ifdef NDEBUG
    #error You forgot to delete NDEBUG !
#endif
```

14.8 #pragma

Syntaxe

```
#pragma compiler specific extension
```

C est un langage de haut niveau et travaille en général indépendamment de la machine. Dans certains cas, il est cependant inévitable d'effectuer certaines adaptations aux spécificités de la machine. A cet effet, on utilise la directive `#pragma` suivie d'une directive spéciale propre à l'environnement utilisé.

La directive

```
#pragma once
// header file code
```

remplace par exemple le contrôle d'inclusion classique suivant :

```
#ifndef _FILE_NAME_H_
#define _FILE_NAME_H_

/* code */

#endif // #ifndef _FILE_NAME_H_
```

La directive

```
#pragma warning (disable : 4018 )
```

a comme effet, que le warning numéro 4018 est supprimé.

Pour plus d'information concernant la directive `#pragma`, il est recommandé de consulter la documentation du compilateur utilisé.

Attention

La directive `#pragma` n'est pas portable dans tous les cas !